

CSP571 Data Preparation and Analysis Spring 2025

April 30, 2025

Final Project Report

Title: The Data-Driven Commute: How Cab Analytics Improve City Mobility

Project Group

Anushka Sarath	A20576979	asarath@hawk.iit.edu
Lokesh Sambireddygar	A20536430	lsambireddygar@hawk.iit.edu
Anantha Sai Valluru	A20544418	avalluru1@hawk.iit.edu

Project Group Leader:

Anushka Sarath	A20576979	asarath@hawk.iit.edu
----------------	-----------	--

1 Abstract

The predictive methods used to estimate ride-hailing pricing for two significant service providers—Uber and Lyft—are thoroughly examined in this paper. We sought to build models that accurately anticipate price based on a range of features, such as time, distance, weather, and service type, using methods such as Random Forest, Decision Trees, and Linear Regression. Understanding dynamic pricing systems and giving users a tool to predict ride costs were the goals.

Uber's pricing structure benefited greatly from the simplicity and efficiency provided by linear regression models, which took use of the strong linear link between features and pricing. Though they had problems with model overfitting, which significantly affected their performance for Lyft, decision trees offered insightful information about feature splits and decision-making logic. The best model for Lyft turned out to be Random Forest, which handled intricate feature interactions well and was resilient to overfitting.

The results of the investigation showed the subtle variations between the two organizations' pricing models as well as the advantages and disadvantages of each prediction method. Based on the precision, intricacy, and interpretability needs of their particular applications, these insights help users choose models.

To verify dependability and comprehend their usefulness, the prediction models were assessed using a variety of performance indicators. Based on the empirical findings, this study makes

recommendations for the proper application of each model and lays out a roadmap for future improvements.

2 Overview

Problem Statement: With an emphasis on how outside variables like weather affect pricing tactics and ride demand, this research attempts to examine the dynamics of pricing and service patterns between Uber and Lyft, two ride-sharing services.

Literature Review

Urban mobility has been drastically altered by the growth of ride-sourcing services, especially Uber and Lyft. As a result, numerous scholarly studies have been conducted to examine their operational dynamics, market effects, and relationships with city infrastructure. The results of current empirical research on these services in various urban contexts are compiled in this review.

Shashank H. (2020) explores the data generated by users of Uber and Lyft, emphasizing the enormous amount of daily travel and the resulting data that is accessible for study. The study predicts ride fares by combining a number of variables, such as weather and surge pricing dynamics, using machine learning algorithms in conjunction with linear and logistic regression models. By giving customers precise fare estimates before the start of the ride, this strategy seeks to improve urban commuters' decision-making processes (Shashank H., 2020).

Jiang et al. (2018) provide a thorough comparison of Uber, Lyft, and conventional taxi services, furthering the topic of ride-sourcing services' operational tactics. Specifically in the contexts of New York City and San Francisco, their research focuses on important market factors like supply, demand, pricing, and wait times. The study clarifies the accessibility of vehicle-for-hire (VFH) services and their connection to socioeconomic aspects and urban transportation infrastructures by utilizing spatial lag models and point pattern statistics (Jiang et al., 2018).

By offering a thorough examination of Uber and Lyft marketing tactics and their effects on the ride-hailing ecosystem, Jingyu Sun (2022) adds to this corpus of work. The study highlights how surge pricing is dynamic and how it affects driver supply and customer behavior. Sun's research highlights the technological and legal obstacles that ride-sourcing platforms must overcome while providing insights into the competitive environment of these services and how they differ from regular taxis in terms of operations (Sun, 2018).

3 Data Processing

Data Processing and Pipeline:

The goal of our data pre-processing strategy is to guarantee the dataset's usability and integrity for precise modeling and analysis. Throughout the pre-processing stage, we have addressed a number of important issues:

Handling Missing Values and Inconsistencies:

The dataset was problematic due to a large number of missing values and inconsistent column labels between files. This was especially evident in source and destination names, where a large number of missing values could greatly distort the analysis's findings. In order to solve these problems and guarantee consistency and completeness throughout the dataset, we have standardized the naming conventions and fixed any missing data.

Standardizing Date-Time Formats:

The merging and analysis procedures were made more difficult by the multiple date-time formats we came across in the various files. We have simplified the dataset and made integration and analysis easier by standardizing these formats.

Data Type Discrepancies:

Errors in integration and analysis may result from differences in data type values for the same features found in several files. To avoid any inconsistencies during the data processing stage, we have standardized these data types across all files.

Feature Engineering and Modification:

Using the journey's "started time" and "ended time," we computed and inserted the trip duration wherever it was lacking. To give the study greater depth, additional columns that indicated the day of the week were generated from the date field. To guarantee uniformity, we made sure that all files used the same standard format for date-time and other feature data. Columns like SunriseTime, SunsetTime, and MoonPhase that were not shared by all files were removed in order to preserve dataset homogeneity. To improve the completeness of the data, latitude and longitude values were used to fill in the missing source and destination information.

Integrating Weather Data:

Inaccurate correlations between weather and travel data may result from the weather data's distinct date format from the main datasets. To ensure accuracy and consistency in upcoming analyses, we have changed the weather dataset's date format to match the main dataset's. Following data preprocessing, the 23 pertinent features are as follows:

4 Data Analysis

Following data preprocessing, the 23 pertinent features are as follows:

Fri	Lyft XL	UberPool	Partly Cloudy
Sat	Lux Black XL	UberXL	WAV
Sun	Lux Black	Black	Possible Drizzle
Shared	surge_multiplier	Black SUV	Overcast
Mostly Cloudy	Drizzle	Rain	Light Rain
distance	Partly Cloudy	Foggy	-

```
In [1]: # Install packages from CRAN
install.packages(c("caret", "sqldf", "tidyr", "tidyverse", "ggplot2", "readr", "gmodels", "tm",
                  "SnowballC", "wordcloud", "RColorBrewer", "treemap", "highcharter", "remotes",
                  "corrplot", "rpart.plot", "magrittr", "dplyr", "igraph", "pillar", "repr",
                  "lifecycle", "ellipsis", "htmltools", "vctrs", "lubridate", "Rtsne", "umap", "rlang"))

# Install packages from specific sources or GitHub
remotes::install_github("jbkunst/highcharter")

# Load Libraries
library(caret)
library(sqldf)
library(tidyr)
library(tidyverse)
library(ggplot2)
library(readr)
library(gmodels)
library(tm)
library(SnowballC)
library(wordcloud)
library(RColorBrewer)
library(treemap)
library(highcharter)
library(corrplot)
library(rpart.plot)
library(magrittr)
library(dplyr)
library(igraph)
library(pillar)
library(repr)
library(lifecycle)
library(ellipsis)
library(htmltools)
```

```
library(vctr)
library(lubridate)
library(Rtsne)
library(umap)
```

```
Installing packages into 'C:/Users/ANUSHKA/AppData/Local/R/win-library/4.5'
(as 'lib' is unspecified)
```

```
package 'caret' successfully unpacked and MD5 sums checked
package 'sqldf' successfully unpacked and MD5 sums checked
package 'tidyr' successfully unpacked and MD5 sums checked
package 'tidyverse' successfully unpacked and MD5 sums checked
package 'ggplot2' successfully unpacked and MD5 sums checked
package 'readr' successfully unpacked and MD5 sums checked
package 'gmodels' successfully unpacked and MD5 sums checked
package 'tm' successfully unpacked and MD5 sums checked
package 'SnowballC' successfully unpacked and MD5 sums checked
package 'wordcloud' successfully unpacked and MD5 sums checked
package 'RColorBrewer' successfully unpacked and MD5 sums checked
package 'treemap' successfully unpacked and MD5 sums checked
package 'highcharter' successfully unpacked and MD5 sums checked
package 'remotes' successfully unpacked and MD5 sums checked
package 'corrplot' successfully unpacked and MD5 sums checked
package 'rpart.plot' successfully unpacked and MD5 sums checked
```

```
[2]: install.packages("rlang")
```

```
Installing package into 'C:/Users/ANUSHKA/AppData/Local/R/win-library/4.5'
(as 'lib' is unspecified)
```

```
package 'rlang' successfully unpacked and MD5 sums checked
```

```
Warning message:
"cannot remove prior installation of package 'rlang'"
Warning message in file.copy(savedcopy, lib, recursive = TRUE):
```

```
In [2]: install.packages("rlang")
```

```
Installing package into 'C:/Users/ANUSHKA/AppData/Local/R/win-library/4.5'
(as 'lib' is unspecified)
```

```
package 'rlang' successfully unpacked and MD5 sums checked
```

```
Warning message:
"cannot remove prior installation of package 'rlang'"
Warning message in file.copy(savedcopy, lib, recursive = TRUE):
"problem copying C:/Users/ANUSHKA/AppData/Local/R/win-library/4.5/00LOCK/rlang/libs/x64/rlang.dll to C:/Users/ANUSHKA/AppData/Local/R/win-library/4.5/rlang/libs/x64/rlang.dll: Permission denied"
Warning message:
"restored 'rlang'"
```

```
The downloaded binary packages are in
C:/Users/ANUSHKA/AppData/Local/Temp/Rtmp8aIcOH/downloaded_packages
```

```
In [6]: install.packages("dplyr")
```

```
Warning message:
"package 'dplyr' is in use and will not be installed"
```

```
In [1]: library(dplyr)
```

```
Attaching package: 'dplyr'
```

```
The following objects are masked from 'package:stats':
```

```
In [2]: cabDataSet <- read.csv("rideshare_kaggle.csv")
```

```
In [3]: summary(cabDataSet)
```

```

      id      timestamp      hour      day
Length:693071  Min.   :1.543e+09  Min.   : 0.00  Min.   : 1.00
Class :character 1st Qu.:1.543e+09 1st Qu.: 6.00 1st Qu.:13.00
Mode  :character Median :1.544e+09 Median :12.00 Median :17.00
              Mean  :1.544e+09 Mean  :11.62 Mean  :17.79
              3rd Qu.:1.545e+09 3rd Qu.:18.00 3rd Qu.:28.00
              Max.   :1.545e+09 Max.   :23.00 Max.   :30.00

      month      datetime      timezone      source
Min.   :11.00  Length:693071  Length:693071  Length:693071
1st Qu.:11.00  Class :character  Class :character  Class :character
Median :12.00  Mode  :character  Mode  :character  Mode  :character
Mean   :11.59
3rd Qu.:12.00
Max.   :12.00

destination      cab_type      product_id      name
Length:693071    Length:693071    Length:693071    Length:693071
Class :character  Class :character  Class :character  Class :character

```

```
In [3]: summary(cabDataSet)
```

```

destination      cab_type      product_id      name
Length:693071    Length:693071    Length:693071    Length:693071
Class :character  Class :character  Class :character  Class :character
Mode  :character  Mode  :character  Mode  :character  Mode  :character

```

```

      price      distance      surge_multiplier      latitude
Min.   : 2.50  Min.   :0.020  Min.   :1.000  Min.   :42.21
1st Qu.: 9.00 1st Qu.:1.280 1st Qu.:1.000 1st Qu.:42.35
Median :13.50 Median :2.160 Median :1.000 Median :42.35
Mean   :16.55 Mean   :2.189 Mean   :1.014 Mean   :42.34
3rd Qu.:22.50 3rd Qu.:2.920 3rd Qu.:1.000 3rd Qu.:42.36
Max.   :97.50 Max.   :7.860 Max.   :3.000 Max.   :42.37
NA's   :55095

```

```

      longitude      temperature      apparentTemperature      short_summary
Min.   :-71.11  Min.   :18.91  Min.   :12.13  Length:693071
1st Qu.: -71.08 1st Qu.:26.45 1st Qu.:21.01  Class :character

```

In [3]: summary(cabDataSet)

```
Max.      :-71.03   Max.      :57.22   Max.      :57.22

long_summary      precipIntensity      precipProbability      humidity
Length:693071     Min.      :0.000000     Min.      :0.0000     Min.      :0.3800
Class :character   1st Qu.:0.000000     1st Qu.:0.0000     1st Qu.:0.6400
Mode  :character   Median :0.000000     Median :0.0000     Median :0.7100
                        Mean      :0.008922     Mean      :0.1461     Mean      :0.7411
                        3rd Qu.:0.000000     3rd Qu.:0.0000     3rd Qu.:0.8800
                        Max.      :0.144700     Max.      :1.0000     Max.      :0.9600

      windSpeed      windGust      windGustTime      visibility
Min.      : 0.450     Min.      : 0.80     Min.      :1.543e+09     Min.      : 0.717
1st Qu.: 3.410     1st Qu.: 4.06     1st Qu.:1.543e+09     1st Qu.: 8.432
Median : 5.910     Median : 7.55     Median :1.544e+09     Median : 9.880
Mean      : 6.186     Mean      : 8.47     Mean      :1.544e+09     Mean      : 8.468
3rd Qu.: 8.410     3rd Qu.:11.74     3rd Qu.:1.545e+09     3rd Qu.: 9.996
Max.      :15.000     Max.      :27.25     Max.      :1.545e+09     Max.      :10.000

temperatureHigh temperatureHighTime temperatureLow temperatureLowTime
Min.      :32.68     Min.      :1.543e+09     Min.      :17.85     Min.      :1.543e+09
```

In [3]: summary(cabDataSet)

```
temperatureHigh temperatureHighTime temperatureLow temperatureLowTime
Min.      :32.68     Min.      :1.543e+09     Min.      :17.85     Min.      :1.543e+09
1st Qu.:42.57     1st Qu.:1.543e+09     1st Qu.:30.17     1st Qu.:1.543e+09
Median :44.68     Median :1.544e+09     Median :34.18     Median :1.544e+09
Mean      :45.04     Mean      :1.544e+09     Mean      :34.15     Mean      :1.544e+09
3rd Qu.:46.91     3rd Qu.:1.545e+09     3rd Qu.:38.73     3rd Qu.:1.545e+09
Max.      :57.87     Max.      :1.545e+09     Max.      :46.60     Max.      :1.545e+09

apparentTemperatureHigh apparentTemperatureHighTime apparentTemperatureLow
Min.      :22.62           Min.      :1.543e+09           Min.      :11.81
1st Qu.:36.57           1st Qu.:1.543e+09           1st Qu.:27.70
Median :40.95           Median :1.544e+09           Median :30.03
Mean      :41.61           Mean      :1.544e+09           Mean      :30.14
3rd Qu.:44.12           3rd Qu.:1.545e+09           3rd Qu.:35.32
Max.      :57.20           Max.      :1.545e+09           Max.      :47.25

apparentTemperatureLowTime      icon      dewPoint      pressure
Min.      :1.543e+09      Length:693071      Min.      : 4.39     Min.      : 988.1
1st Qu.:1.543e+09      Class :character   1st Qu.:27.49     1st Qu.: 999.8
Median :1.544e+09      Mode  :character   Median :30.69     Median :1009.2
```

```
In [3]: summary(cabDataSet)
```

```
      windBearing      cloudCover      uvIndex      visibility.1
Min.   : 2.0      Min.   :0.0000      Min.   :0.0000      Min.   : 0.717
1st Qu.:124.0      1st Qu.:0.3700      1st Qu.:0.0000      1st Qu.: 8.432
Median :258.0      Median :0.8200      Median :0.0000      Median : 9.880
Mean   :220.1      Mean   :0.6865      Mean   :0.2489      Mean   : 8.468
3rd Qu.:303.0      3rd Qu.:1.0000      3rd Qu.:0.0000      3rd Qu.: 9.996
Max.   :356.0      Max.   :1.0000      Max.   :2.0000      Max.   :10.000

      ozone      sunriseTime      sunsetTime      moonPhase
Min.   :269.4      Min.   :1.543e+09      Min.   :1.543e+09      Min.   :0.0900
1st Qu.:290.9      1st Qu.:1.543e+09      1st Qu.:1.543e+09      1st Qu.:0.3000
Median :307.4      Median :1.544e+09      Median :1.544e+09      Median :0.6800
Mean   :313.5      Mean   :1.544e+09      Mean   :1.544e+09      Mean   :0.5791
3rd Qu.:331.8      3rd Qu.:1.545e+09      3rd Qu.:1.545e+09      3rd Qu.:0.7900
Max.   :378.9      Max.   :1.545e+09      Max.   :1.545e+09      Max.   :0.9300

precipIntensityMax uvIndexTime      temperatureMin temperatureMinTime
Min.   :0.00000      Min.   :1.543e+09      Min.   :15.63      Min.   :1.543e+09
1st Qu.:0.00000      1st Qu.:1.543e+09      1st Qu.:30.17      1st Qu.:1.543e+09
```

```
In [3]: summary(cabDataSet)
```

```
      3rd Qu.:0.09160      3rd Qu.:1.545e+09      3rd Qu.:38.88      3rd Qu.:1.545e+09
Max.   :0.14590      Max.   :1.545e+09      Max.   :43.10      Max.   :1.545e+09

temperatureMax temperatureMaxTime apparentTemperatureMin
Min.   :33.51      Min.   :1.543e+09      Min.   :11.81
1st Qu.:42.57      1st Qu.:1.543e+09      1st Qu.:27.76
Median :44.68      Median :1.544e+09      Median :30.13
Mean   :45.26      Mean   :1.544e+09      Mean   :29.73
3rd Qu.:46.91      3rd Qu.:1.545e+09      3rd Qu.:35.71
Max.   :57.87      Max.   :1.545e+09      Max.   :40.05

apparentTemperatureMinTime apparentTemperatureMax apparentTemperatureMaxTime
Min.   :1.543e+09      Min.   :28.95      Min.   :1.543e+09
1st Qu.:1.543e+09      1st Qu.:36.57      1st Qu.:1.543e+09
Median :1.544e+09      Median :40.95      Median :1.544e+09
Mean   :1.544e+09      Mean   :42.00      Mean   :1.544e+09
3rd Qu.:1.545e+09      3rd Qu.:44.12      3rd Qu.:1.545e+09
Max.   :1.545e+09      Max.   :57.20      Max.   :1.545e+09
```



```
In [4]: # Basic overview
totalRows <- nrow(cabDataSet)
totalCols <- ncol(cabDataSet)

summaryInfo <- data.frame(
  DatasetName      = "rideshare_kaggle.csv",
  DatasetDescription = "Uber and Lyft ride records for Boston, MA",
  Record_Count     = totalRows,
  Attribute_Count   = totalCols
)
```

```
In [5]: print(summaryInfo)
```

	DatasetName	DatasetDescription	Record_Count
1	rideshare_kaggle.csv	Uber and Lyft ride records for Boston, MA	693071
	Attribute_Count		
1			57

```
In [6]: # Display the summary information
View(summaryInfo)
```

```
View(summaryInfo)
```

A data.frame: 1 × 4

	DatasetName	DatasetDescription	Record_Count	Attribute_Count
	<chr>	<chr>	<int>	<int>
1	rideshare_kaggle.csv	Uber and Lyft ride records for Boston, MA	693071	57

```
In [7]: str(cabDataSet)
```

```
'data.frame': 693071 obs. of 57 variables:
 $ id          : chr "424553bb-7174-41ea-aeb4-fe06d4f4b9d7" "4bd23055-6827-41c6-b23b-3c491f24e74d" "981a3613-77af-4620-a42a-0c0866077d1e" "c2d88af2-d278-4bfd-a8d0-29ca77cc5512" ...
 $ timestamp   : num 1.54e+09 1.54e+09 1.54e+09 1.54e+09 1.54e+09 ...
 $ hour        : int 9 2 1 4 3 18 5 19 6 10 ...
 $ day         : int 16 27 28 30 29 17 26 2 3 27 ...
 $ month       : int 12 11 11 11 11 12 11 12 12 11 ...
 $ datetime    : chr "2018-12-16 09:30:07" "2018-11-27 02:00:23" "2018-11-28 01:00:22" "2018-11-30 04:53:02"
 ...
 $ timezone    : chr "America/New_York" "America/New_York" "America/New_York" "America/New_York" ...
 $ source      : chr "Haymarket Square" "Haymarket Square" "Haymarket Square" "Haymarket Square" ...
 $ destination : chr "North Station" "North Station" "North Station" "North Station" ...
 $ cab_type    : chr "Lyft" "Lyft" "Lyft" "Lyft" ...
 $ product_id  : chr "lyft_line" "lyft_premier" "lyft" "lyft_luxsuv" ...
 $ name        : chr "Shared" "Lux" "Lyft" "Lux Black XL" ...
 $ price       : num 5 11 7 26 9 16.5 10.5 16.5 3 27.5 ...
 $ distance    : num 0.44 0.44 0.44 0.44 0.44 0.44 0.44 1.08 1.08 1.08 ...
 $ surge_multiplier : num 1 1 1 1 1 1 1 1 1 ...
 $ latitude    : num 42.2 42.2 42.2 42.2 42.2 ...
 $ longitude   : num -71 -71 -71 -71 -71 ...
 $ temperature : num 42.3 43.6 38.3 34.4 37.4 ...
 $ apparentTemperature : num 37.1 37.4 32.9 29.6 30.9 ...
 $ short_summary : chr " Mostly Cloudy " " Rain " " Clear " " Clear " ...
 $ long_summary : chr " Rain throughout the day. " " Rain until morning, starting again in the evening. " " Light rain in the morning. " " Mostly cloudy throughout the day. "
```

```

# source_multiplier : num 1 1 1 1 1 1 1 1 ...
$ latitude : num 42.2 42.2 42.2 42.2 42.2 ...
$ longitude : num -71 -71 -71 -71 -71 ...
$ temperature : num 42.3 43.6 38.3 34.4 37.4 ...
$ apparentTemperature : num 37.1 37.4 32.9 29.6 30.9 ...
$ short_summary : chr " Mostly Cloudy " " Rain " " Clear " " Clear " ...
$ long_summary : chr " Rain throughout the day. " " Rain until morning, starting again in the evening. " " Light rain in the morning. " " Partly cloudy throughout the day. " ...
$ precipIntensity : num 0 0.13 0 0 0 ...
$ precipProbability : num 0 1 0 0 0 0 1 0 1 ...
$ humidity : num 0.68 0.94 0.75 0.73 0.7 0.84 0.91 0.93 0.96 0.93 ...
$ windSpeed : num 8.66 11.98 7.33 5.28 9.14 ...
$ windGust : num 9.17 11.98 7.33 5.28 9.14 ...
$ windGustTime : int 1545015600 1543291200 1543334400 1543514400 1543446000 1545022800 1543287600 1543755600 15
43856400 1543338000 ...
$ visibility : num 10 4.79 10 10 10 ...
$ temperatureHigh : num 43.7 47.3 47.5 45 42.2 ...
$ temperatureHighTime : int 1544968800 1543251600 1543320000 1543510800 1543420800 1545076800 1543255200 1543788000 15
43852800 1543320000 ...
$ temperatureLow : num 34.2 42.1 33.1 28.9 36.7 ...
$ temperatureLowTime : int 1545048000 1543298400 1543402800 1543579200 1543478400 1545130800 1543298400 1543816800 15
43921200 1543399200 ...
$ apparentTemperatureHigh : num 38 43.9 44.1 38.5 35.8 ...
$ apparentTemperatureHighTime : int 1544968800 1543251600 1543320000 1543510800 1543420800 1545080400 1543251600 1543788000 15
43852800 1543320000 ...
$ apparentTemperatureLow : num 27.4 36.2 29.1 26.2 30.3 ...
$ apparentTemperatureLowTime : int 1545044400 1543291200 1543392000 1543575600 1543460400 1545134400 1543298400 1543816800 15
43914000 1543399200 ...
$ icon : chr " partly-cloudy-night " " rain " " clear-night " " clear-night " ...
$ dewPoint : num 32.7 41.8 31.1 26.6 28.6 ...
$ pressure : num 1022 1004 992 1014 998 ...
$ windBearing : int 57 90 240 310 303 294 91 159 307 79 ...
$ cloudCover : num 0.72 1 0.03 0 0.44 1 1 1 1 1 ...
$ uvIndex : int 0 0 0 0 0 1 0 0 0 0 ...
$ visibility.1 : num 10 4.79 10 10 10 ...
$ ozone : num 304 291 316 291 348 ...
$ sunriseTime : int 1544962084 1543232969 1543319437 1543492370 1543405904 1545048523 1543233004 1543751798 15
43852800 1543320000 ...
$ apparentTemperatureLow : num 27.4 36.2 29.1 26.2 30.3 ...
$ apparentTemperatureLowTime : int 1545044400 1543291200 1543392000 1543575600 1543460400 1545134400 1543298400 1543816800 15
43914000 1543399200 ...
$ icon : chr " partly-cloudy-night " " rain " " clear-night " " clear-night " ...
$ dewPoint : num 32.7 41.8 31.1 26.6 28.6 ...
$ pressure : num 1022 1004 992 1014 998 ...
$ windBearing : int 57 90 240 310 303 294 91 159 307 79 ...
$ cloudCover : num 0.72 1 0.03 0 0.44 1 1 1 1 1 ...
$ uvIndex : int 0 0 0 0 0 1 0 0 0 0 ...
$ visibility.1 : num 10 4.79 10 10 10 ...
$ ozone : num 304 291 316 291 348 ...
$ sunriseTime : int 1544962084 1543232969 1543319437 1543492370 1543405904 1545048523 1543233004 1543751798 15
43838259 1543319472 ...
$ sunsetTime : int 1544994864 1543266992 1543353364 1543526114 1543439738 1545081282 1543266980 1543785242 15
43871628 1543353352 ...
$ moonPhase : num 0.3 0.64 0.68 0.75 0.72 0.33 0.64 0.86 0.89 0.68 ...
$ precipIntensityMax : num 0.1276 0.13 0.1064 0 0.0001 ...
$ uvIndexTime : int 1544979600 1543251600 1543338000 1543507200 1543420800 1545066000 1543251600 1543770000 15
43852800 1543338000 ...
$ temperatureMin : num 39.9 40.5 35.4 34.7 33.1 ...
$ temperatureMinTime : int 1545012000 1543233600 1543377600 1543550400 1543402800 1545048000 1543233600 1543726800 15
43896000 1543377600 ...
$ temperatureMax : num 43.7 47.3 47.5 45 42.2 ...
$ temperatureMaxTime : int 1544968800 1543251600 1543320000 1543510800 1543420800 1545022800 1543255200 1543788000 15
43852800 1543320000 ...
$ apparentTemperatureMin : num 33.7 36.2 31 30.3 29.1 ...
$ apparentTemperatureMinTime : int 1545012000 1543291200 1543377600 1543550400 1543392000 1545044400 1543291200 1543748400 15
43896000 1543377600 ...
$ apparentTemperatureMax : num 38.1 43.9 44.1 38.5 35.8 ...
$ apparentTemperatureMaxTime : int 1544958000 1543251600 1543320000 1543510800 1543420800 1545080400 1543251600 1543788000 15
43852800 1543320000 ...

```

```
In [8]: # Feature metadata
featureInfo <- data.frame(
  FeatureName = c("price", "distance", "time_of_day"),
  DataClass   = c("Numeric", "Numeric", "Character"),
  Notes       = c("Ride fare in USD", "Ride length in miles", "Time of day the ride started")
)
```

```
In [9]: # Print the feature metadata
print(featureInfo)
```

```
FeatureName DataClass      Notes
1      price  Numeric      Ride fare in USD
2    distance  Numeric      Ride length in miles
3 time_of_day Character Time of day the ride started
```

```
In [10]: colnames(cabDataSet)
```

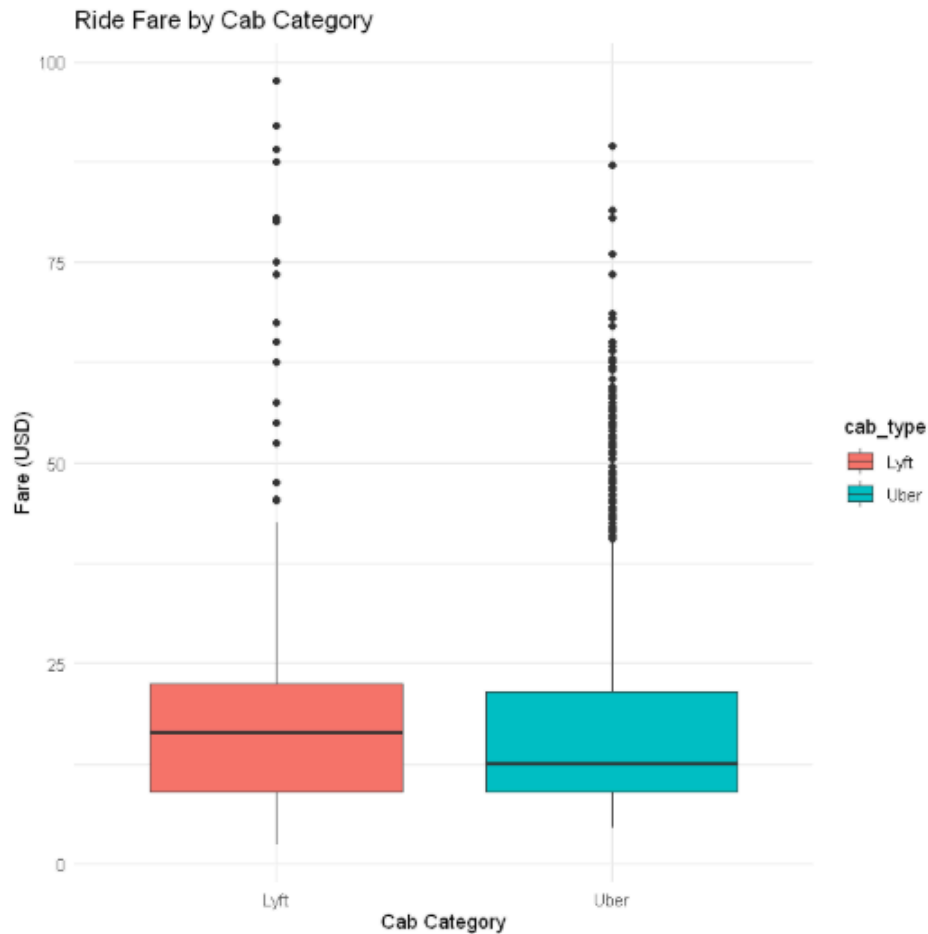
```
'id' 'timestamp' 'hour' 'day' 'month' 'datetime' 'timezone' 'source' 'destination' 'cab_type' 'product_id' 'name' 'price' 'distance' 'surge_multiplier'
'latitude' 'longitude' 'temperature' 'apparentTemperature' 'short_summary' 'long_summary' 'precipIntensity' 'precipProbability' 'humidity' 'windSpeed'
'windGust' 'windGustTime' 'visibility' 'temperatureHigh' 'temperatureHighTime' 'temperatureLow' 'temperatureLowTime' 'apparentTemperatureHigh'
'apparentTemperatureHighTime' 'apparentTemperatureLow' 'apparentTemperatureLowTime' 'icon' 'dewPoint' 'pressure' 'windBearing' 'cloudCover'
'uvIndex' 'visibility.1' 'ozone' 'sunriseTime' 'sunsetTime' 'moonPhase' 'precipIntensityMax' 'uvIndexTime' 'temperatureMin' 'temperatureMinTime'
'temperatureMax' 'temperatureMaxTime' 'apparentTemperatureMin' 'apparentTemperatureMinTime' 'apparentTemperatureMax'
'apparentTemperatureMaxTime'
```

Data Stylized Facts:

```
In [11]: library(ggplot2)

# Boxplot of ride fare by cab category
ggplot(cabDataSet, aes(x = cab_type, y = price, fill = cab_type)) +
  geom_boxplot() +
  labs(
    title = "Ride Fare by Cab Category",
    x     = "Cab Category",
    y     = "Fare (USD)"
  ) +
  theme_minimal()
```

```
Warning message:
"Removed 55095 rows containing non-finite outside the scale range
(`stat_boxplot()`)."
```

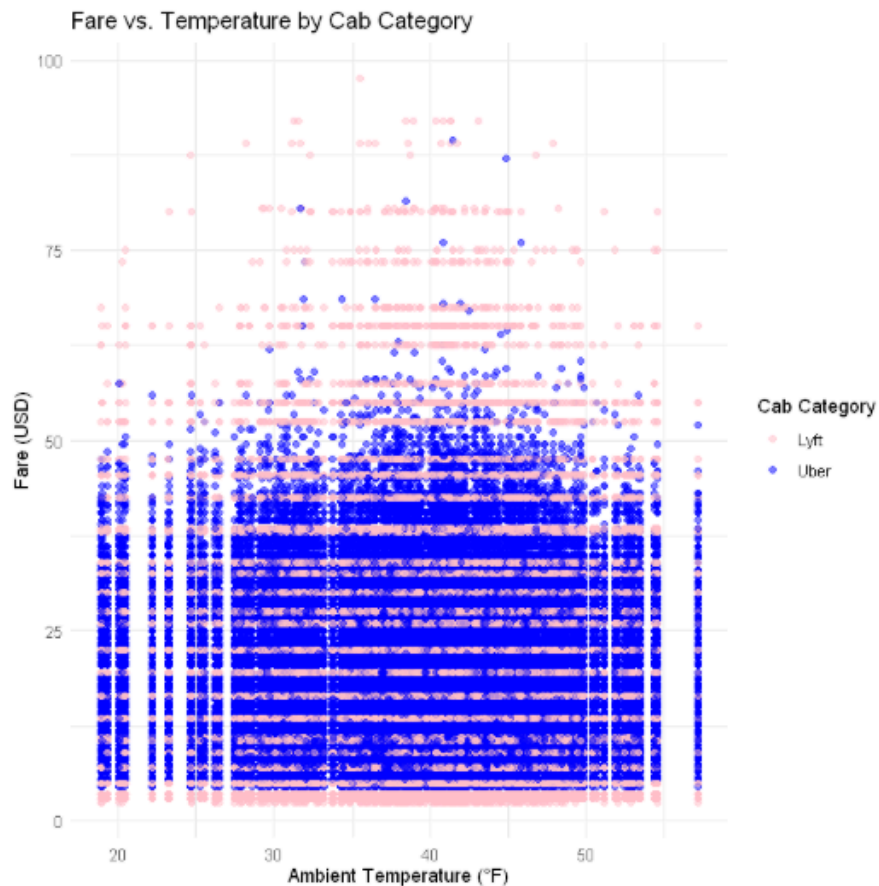


Using a box-and-whisker plot to tastefully depict the fare distribution, the figure above provides a thorough comparison of the pricing structures of Lyft and Uber. Distinct colors—teal for Uber and red for Lyft—are used to indicate the price differences, making it easy to distinguish between the two services visually. It is noteworthy that the plot contains outliers, which are isolated points that indicate fare incidents that substantially vary from the norm. This graph provides useful information for stakeholders making defensible decisions by illuminating the variations in pricing methods and customer cost implications linked to each service.

```
In [13]: library(ggplot2)

# Scatter plot of fare against ambient temperature by cab category
ggplot(cabDataSet, aes(x = temperature, y = price)) +
  geom_point(aes(color = cab_type), alpha = 0.5) +
  labs(
    title = "Fare vs. Temperature by Cab Category",
    x     = "Ambient Temperature (°F)",
    y     = "Fare (USD)",
    color = "Cab Category"
  ) +
  theme_minimal() +
  scale_color_manual(values = c("Uber" = "blue", "Lyft" = "pink"))
```

Warning message:
"Removed 55095 rows containing missing values or values outside the scale range
(`geom\_point()`)."



"Price vs. Temperature by Cab Type" is the title of this scatter graphic. Individual data points are plotted on a graph, with the price in US dollars on the y-axis and the temperature in Fahrenheit on the x-axis. According to the legend, the two distinct colors of the data points—blue and pink—each represent a different kind of taxi: Uber is blue, while Lyft is pink. For both services, the point distribution displays a broad range of costs at various temperatures. While Uber's costs are largely concentrated at the lower end of the price range, Lyft's prices appear to be more dispersed throughout the price range. The x-axis displays temperatures between around 20°F and just over 50°F. Given that the data points are dispersed throughout the temperature range, it appears that there is no discernible relationship between temperature and cost for any type of taxi.

```

In [14]: library(lubridate)
library(ggplot2)

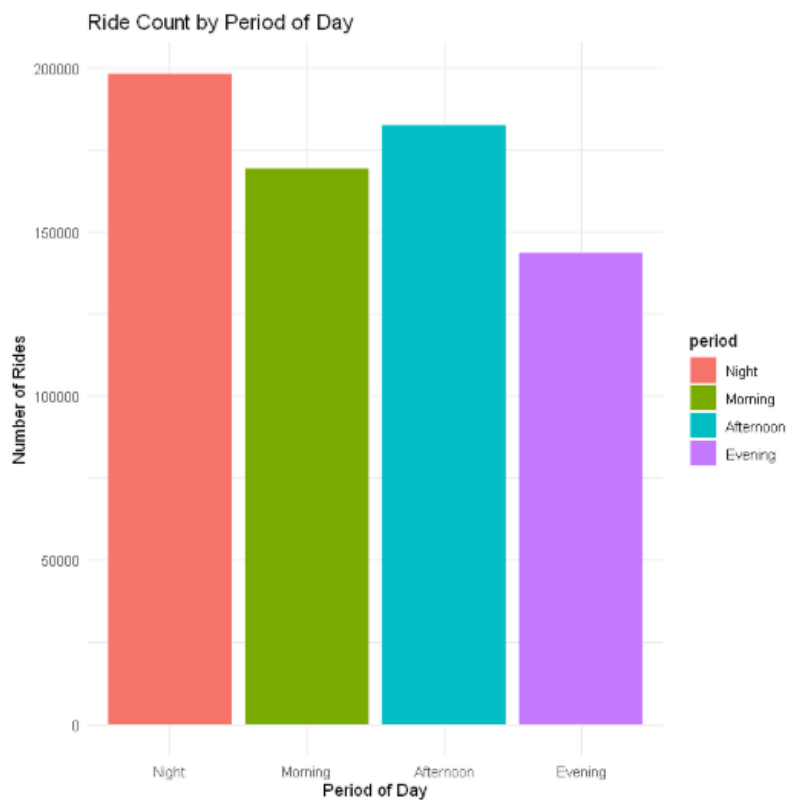
cabDataSet %>%
  mutate(
    ride_time = ymd_hms(datetime), # convert to POSIXct
    ride_hour = hour(ride_time),
    period = cut(
      ride_hour,
      breaks = c(0, 6, 12, 18, 24),
      labels = c("Night", "Morning", "Afternoon", "Evening"),
      include.lowest = TRUE
    )
  ) %>%
  count(period) %>%
  ggplot(aes(x = period, y = n, fill = period)) +
  geom_col() +
  labs(
    title = "Ride Count by Period of Day",
    x = "Period of Day",
    y = "Number of Rides"
  ) +
  theme_minimal()

```

Attaching package: 'lubridate'

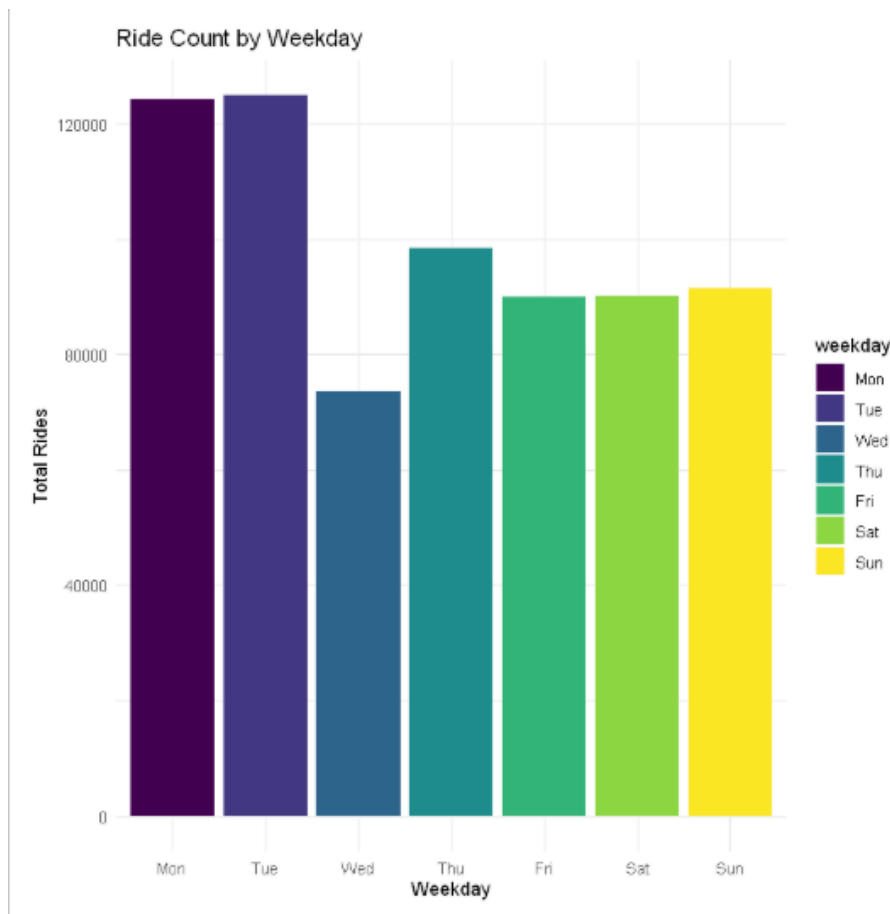
The following objects are masked from 'package:base':

date, intersect, setdiff, union



The aforementioned bar graph, which is divided into night, morning, afternoon, and evening segments, quantifies riding frequencies throughout the day. Because each segment is color-coded, ride-sharing services may quickly determine when they are at their busiest and least busy. Service providers can maximize operational efficiencies and customize their services to satisfy changing market demands by using this visualization, which is an essential tool for monitoring consumer behavior and demand patterns.

```
n [15]: cabDataSet %>%
  mutate(
    ride_ts = ymd_hms(datetime), # ensure POSIXct format
    weekday = wday(ride_ts, label = TRUE, week_start = 1) # Monday = 1
  ) %>%
  count(weekday) %>%
  ggplot(aes(x = weekday, y = n, fill = weekday)) +
  geom_col() +
  labs(
    title = "Ride Count by Weekday",
    x = "Weekday",
    y = "Total Rides"
  ) +
  theme_minimal()
```



The number of rides for each day of the week is displayed in the graph above. Monday is dark purple, Tuesday is a lighter purple, Wednesday is blue, Thursday is green, Friday is light green, Saturday is yellow, and Sunday is light yellow. These colors each stand for a distinct day.

The "Number of Rides" vertical axis begins at 0 and increases by 20,000. Given that they both reach over 120,000 rides, Monday and Tuesday seem to have the greatest bars, indicating that they have the highest riding frequency. There are fewer rides on the other days, and as the week goes on toward Sunday, the number of rides clearly declines. The demand for ride services over the course of the week could be examined using this graph.

5 Exploratory Data Analysis

Which company provides more rides, Lyft or Uber?

```
In [21]: library(dplyr)
library(ggplot2)
library(scales)

# 1. Derive distinct rides by ID
cabDataSet_distinct <- cabDataSet %>%
  distinct(id, .keep_all = TRUE)

# 2. Summarize ride counts and percentages by service
service_summary <- cabDataSet_distinct %>%
  group_by(cab_type) %>%
  summarise(
    total_rides = n(),
    pct_rides   = (n() / nrow(cabDataSet_distinct)) * 100
  )

# 3. Bar chart of ride volume by service provider
ride_volume_plot <- ggplot(cabDataSet_distinct, aes(x = cab_type, fill = cab_type)) +
  geom_bar() +
  scale_y_continuous(
    breaks = seq(0, 1000000, 100000),
    labels = comma
  ) +
  labs(
    title = "Ride Counts by Service Provider",
    x     = "Service Provider",
    y     = "Number of Rides"
  ) +
  theme(
    plot.title       = element_text(size = 15, hjust = 0.5),
    legend.position  = c(1, 1),
    legend.justification = c("right", "top"),
    axis.title       = element_text(size = 12),
    axis.text        = element_text(size = 9).
```



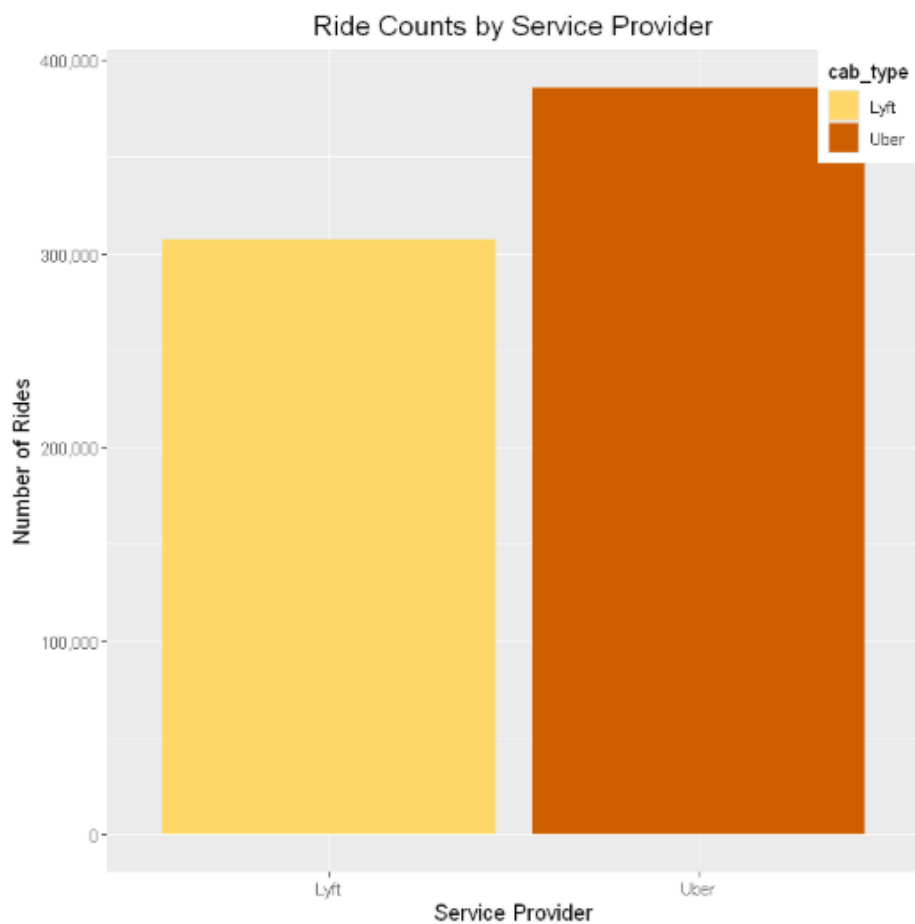
```

service_summary <- cabDataSet_distinct %>%
  group_by(cab_type) %>%
  summarise(
    total_rides = n(),
    pct_rides   = (n() / nrow(cabDataSet_distinct)) * 100
  )

# 3. Bar chart of ride volume by service provider
ride_volume_plot <- ggplot(cabDataSet_distinct, aes(x = cab_type, fill = cab_type)) +
  geom_bar() +
  scale_y_continuous(
    breaks = seq(0, 1000000, 100000),
    labels = comma
  ) +
  labs(
    title = "Ride Counts by Service Provider",
    x     = "Service Provider",
    y     = "Number of Rides"
  ) +
  theme(
    plot.title       = element_text(size = 15, hjust = 0.5),
    legend.position  = c(1, 1),
    legend.justification = c("right", "top"),
    axis.title       = element_text(size = 12),
    axis.text        = element_text(size = 9),
    plot.caption     = element_text(size = 10)
  ) +
  scale_fill_manual(values = c("#FFDB6D", "#D16103"))

# 4. Render the plot
ride_volume_plot

```



Lyft's yellow bar can accommodate up to 175,000 rides. With more than 300,000 rides, Uber's bar is orange. Uber is unquestionably in the lead in the chart that answers the question, "Who offers the most rides?"

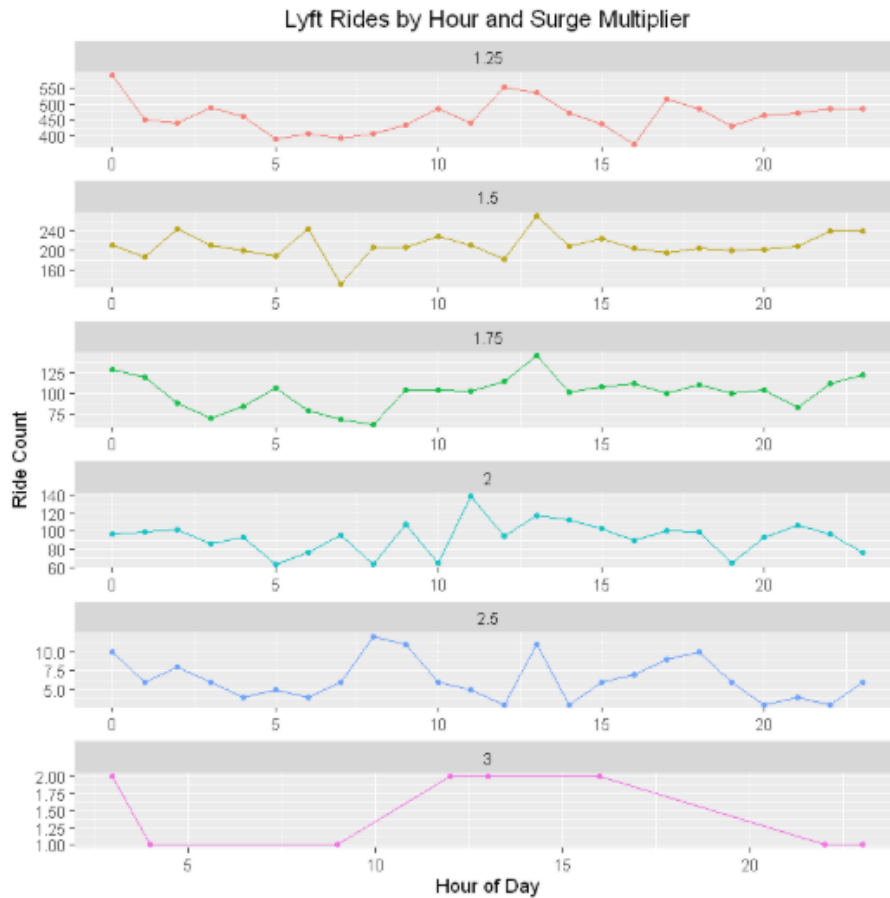
Per-Surge Multiplier for Lyft: Total Rides by Daytime Hour

```
In [22]: # Summarize Lyft surge rides by hour
lyft_surge_summary <- cabDataSet %>%
  filter(cab_type == "Lyft", surge_multiplier > 1) %>%
  group_by(hour, surge_multiplier) %>%
  summarise(
    ride_count = n(),
    .groups = "drop"
  )

# Convert surge multiplier to factor
lyft_surge_summary$surge_multiplier <- as.factor(lyft_surge_summary$surge_multiplier)

# Plot rides by hour for each surge level
lyft_surge_plot <- ggplot(lyft_surge_summary, aes(x = hour, y = ride_count, color = surge_multiplier)) +
  geom_point(alpha = 0.8, size = 1) +
  geom_line() +
  labs(
    title = "Lyft Rides by Hour and Surge Multiplier",
    x = "Hour of Day",
    y = "Ride Count",
    color = "Surge Level"
  ) +
  facet_wrap(~ surge_multiplier, ncol = 1, scales = "free") +
  guides(color = guide_legend(ncol = 1)) +
  theme(
    legend.position = "none",
    panel.border = element_blank(),
    panel.spacing.x = unit(0, "line"),
    plot.title = element_text(hjust = 0.5)
  )

lyft_surge_plot
```



The line graph "Lyft:PerSurgeMultiplier-Total Rides vs. Hour of the Day" is multi-tiered. Each line, which ranges in color from 1.25 to 3, symbolizes a surge multiplier. Depending on the surge multiplier, the lines display the number of rides that fluctuate during the day and peak at various times.

Fare costs, both minimum and maximum

```
In [24]: # install.packages("sqldf")
# install.packages("gmodels")

library(sqldf)
library(gmodels)

# price summaries by route and cab service
route_price_summary <- sqldf("
SELECT
  source,
  destination,
  cab_type,
  AVG(price) AS avg_price,
  MIN(price) AS min_price,
  MAX(price) AS max_price
FROM cabDataSet
GROUP BY source, destination, cab_type
ORDER BY cab_type
")

# contingency table of surge levels vs cab services
CrossTable(
  cabDataSet$surge_multiplier,
  cabDataSet$cab_type
)
```

Loading required package: gsubfn

Loading required package: proto

Loading required package: RSQLite

Cell Contents	
	N
Chi-square contribution	
N / Row Total	
N / Col Total	
N / Table Total	

Total Observations in Table: 693071

cabDataSet\$surge_multiplier	cabDataSet\$cab_type		Row Total
	Lyft	Uber	
1	286433	385663	672096
	456.978	364.253	
	0.426	0.574	0.970
	0.932	1.000	
	0.413	0.556	
1.25	11085	0	11085
	7738.535	6168.307	
	1.000	0.000	0.016
	0.036	0.000	
	0.016	0.000	
1.5	5065	0	5065
	3535.921	2818.446	
	1.000	0.000	0.007
	0.016	0.000	
	0.007	0.000	
1.75	2420	0	2420

1	456.978 0.426 0.932 0.413	364.253 0.574 1.000 0.556	0.970
1.25	11085 7738.535 1.000 0.036 0.016	0 6168.307 0.000 0.000 0.000	11085 0.016
1.5	5065 3535.921 1.000 0.016 0.007	0 2818.446 0.000 0.000 0.000	5065 0.007
1.75	2420 1689.423 1.000 0.008 0.003	0 1346.622 0.000 0.000 0.000	2420 0.003
2	2239 1563.065 1.000 0.007 0.003	0 1245.903 0.000 0.000 0.000	2239 0.003
2.5	154 107.509 1.000 0.001 0.000	0 85.694 0.000 0.000 0.000	154 0.000

1.5	5065 3535.921 1.000 0.016 0.007	0 2818.446 0.000 0.000 0.000	5065 0.007
1.75	2420 1689.423 1.000 0.008 0.003	0 1346.622 0.000 0.000 0.000	2420 0.003
2	2239 1563.065 1.000 0.007 0.003	0 1245.903 0.000 0.000 0.000	2239 0.003
2.5	154 107.509 1.000 0.001 0.000	0 85.694 0.000 0.000 0.000	154 0.000
3	12 8.377 1.000 0.000 0.000	0 6.677 0.000 0.000 0.000	12 0.000
Column Total	307408 0.444	385663 0.556	693071

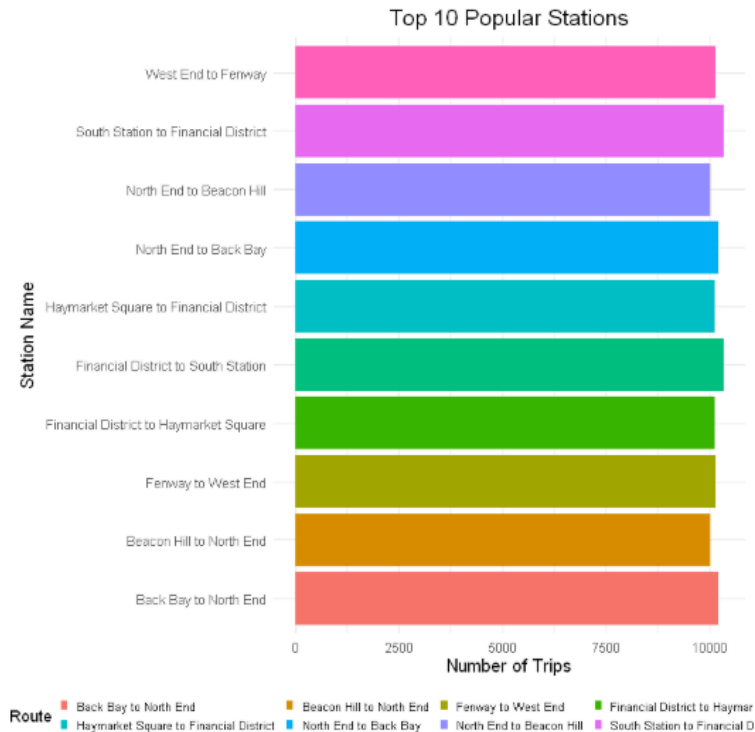
This is a table that compares Uber and Lyft for various surge multipliers. Notably, Lyft gains ground as multipliers rise, indicating a deliberate posture in premium pricing scenarios, while Uber prevails in the no-surge scenario.

The Top 10 Most Well-Known Stations

```
1 [26]: # Top 10 routes by trip count
route_summary <- sqldf("
  SELECT
    source,
    destination,
    (source || ' to ' || destination) AS route_label,
    COUNT(id) AS trip_count
  FROM cabDataSet
  GROUP BY source, destination
  ORDER BY trip_count DESC
  LIMIT 10
")

# Bar chart of most popular routes with legend at bottom
route_popularity_plot <- ggplot(route_summary, aes(x = trip_count, y = route_label, fill = route_label)) +
  geom_col() +
  labs(
    title = "Top 10 Popular Stations",
    x     = "Number of Trips",
    y     = "Station Name",
    fill  = "Route"
  ) +
  theme_minimal() +
  theme(
    plot.title       = element_text(size = 15, hjust = 0.5),
    axis.title       = element_text(size = 12),
    axis.text        = element_text(size = 9),
    legend.position  = "bottom",
    legend.text      = element_text(size = 8),
    legend.key.size  = unit(0.4, "lines")
  ) +
  guides(fill = guide_legend(nrow = 2, byrow = TRUE))

# Render the plot
route_popularity_plot
```



The "Top 10 Popular Stations" horizontal bar chart has distinct colors for each bar. The lengths differ, signifying the quantity of journeys to every station. The pink bar, which is the longest, indicates the most travels from Fenway to West End.

The rides are impacted by the weather

```

1 [28]: # 1. Build and save the summary table
weather_summary <- cabDataSet_distinct %>%
  group_by(short_summary) %>%
  summarise(
    ride_count = n(),
    pct = (n() / nrow(cabDataSet_distinct)) * 100,
    .groups = "drop"
  )

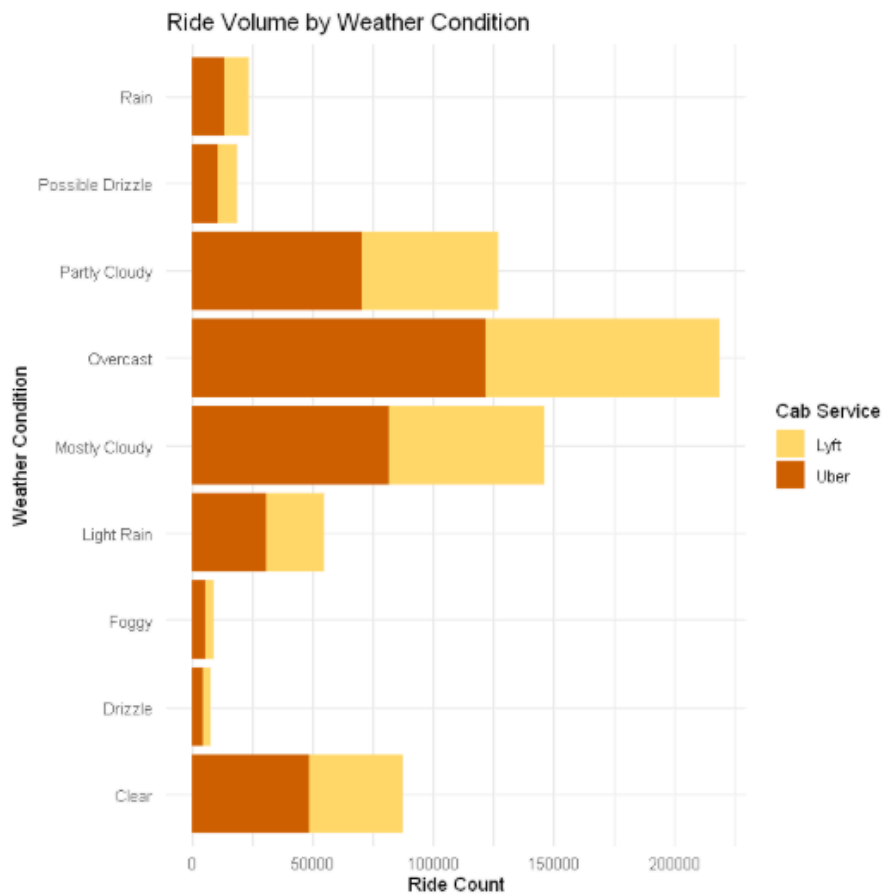
# 2. Print or view the table
print(weather_summary)
# or, in RStudio
# View(weather_summary)

# 3. Then your plot
weather_plot <- ggplot(cabDataSet_distinct, aes(x = short_summary, fill = cab_type)) +
  geom_bar() +
  coord_flip() +
  labs(
    title = "Ride Volume by Weather Condition",
    x = "Weather Condition",
    y = "Ride Count",
    fill = "Cab Service"
  ) +
  theme_minimal() +
  scale_fill_manual(values = c("#FFDB6D", "#D16103"))

weather_plot

```

```
# A tibble: 9 × 3
  short_summary      ride_count  pct
  <chr>          <int> <dbl>
1 " Clear "      87126 12.6
2 " Drizzle "     7296 1.05
3 " Foggy "      9060 1.31
4 " Light Rain "  54912 7.92
5 " Mostly Cloudy " 146210 21.1
6 " Overcast "   218895 31.6
7 " Partly Cloudy " 127224 18.4
8 " Possible Drizzle " 18636 2.69
9 " Rain "      23712 3.42
```



The total number of rides across various weather conditions is displayed in bars with two shades: light for Lyft and darker for Uber. Lyft peaks under "Mostly Cloudy," while Uber sees the most rides on clear days.

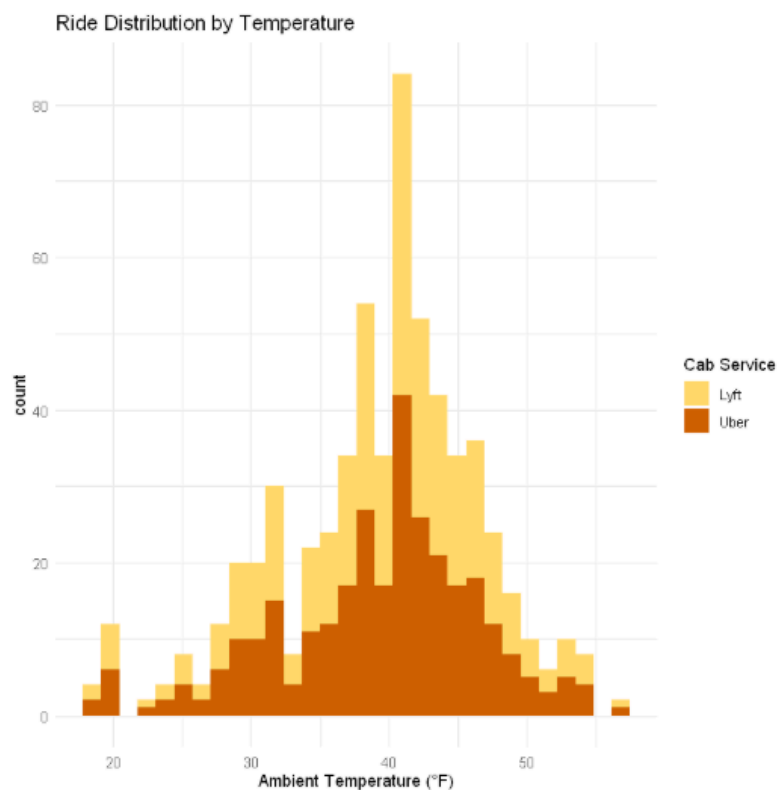
The cost of the ride is impacted by temperature


```
[29]: # Temperature-ride summary by service
temperature_summary <- sqldf("
  SELECT
    temperature,
    price,
    cab_type
  FROM cabDataSet
  GROUP BY cab_type, temperature
")

# Histogram of rides across temperature values
temperature_plot <- temperature_summary %>%
  ggplot(aes(x = temperature, fill = cab_type)) +
  geom_histogram() +
  labs(
    title = "Ride Distribution by Temperature",
    x     = "Ambient Temperature (°F)",
    fill  = "Cab Service"
  ) +
  theme_minimal()

temperature_plot +
  scale_fill_manual(values = c("#FFDB6D", "#D16103"))

`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



Two superimposed histograms are displayed in "Cabs Affected by Temperature," one in orange for Uber and one in yellow for Lyft. The histograms display the number of trips throughout temperature ranges, with Uber showing a discernible jump at about 40°F.

Whether the passengers have access to cabs

```
[31]: # Load required packages
# install.packages("tm")
# install.packages("wordcloud")
# install.packages("RColorBrewer")
library(tm)
library(wordcloud)
library(RColorBrewer)

# Build text corpus
text_corpus <- Corpus(VectorSource(cabDataSet$long_summary))

# Create term-document matrix and convert to matrix
term_matrix <- as.matrix(TermDocumentMatrix(text_corpus))

# Compute and sort term frequencies
freq_vec <- sort(rowSums(term_matrix), decreasing = TRUE)

# Prepare data frame of terms and their frequencies
freq_table <- data.frame(
  term      = names(freq_vec),
  frequency = freq_vec
)

# Generate word cloud
set.seed(3)
wordcloud(
  words = freq_table$term,
  freq  = freq_table$frequency,
  colors = brewer.pal(8, "Dark2")
)
```

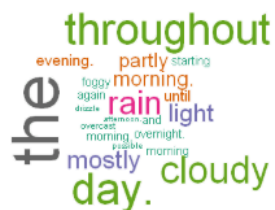
Loading required package: NLP

Attaching package: 'NLP'

The following object is masked from 'package:ggplot2':

annotate

Loading required package: RColorBrewer



The largest words in the word cloud are "throughout" and "day." Other weather-related terms, such as "rain," "mostly," and "cloudy," as well as terms referring to different times of the day, such as "morning" and "evening," fluctuate in size, showing how frequently they occur in a dataset.

Trips Every Hour

```
In [32]: library(dplyr)
library(ggplot2)

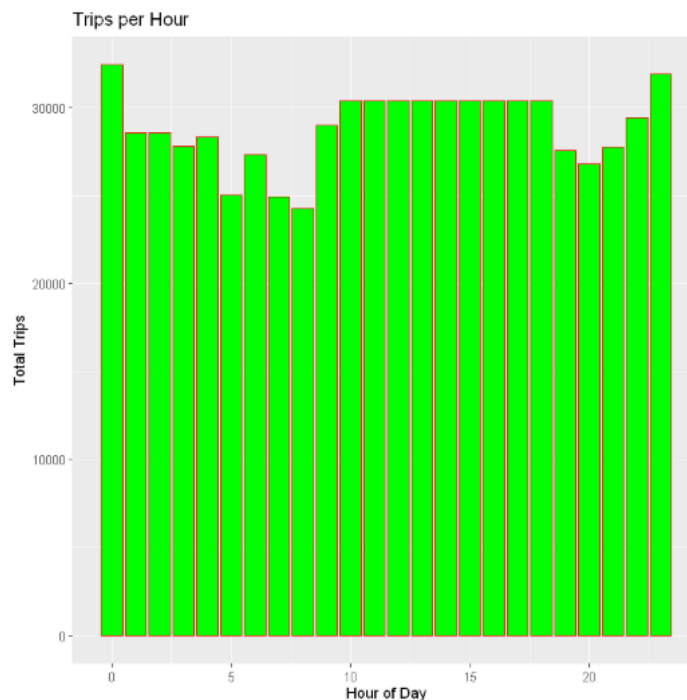
# Summarize total trips by hour
hourly_summary <- cabDataSet %>%
  group_by(hour) %>%
  summarise(
    trip_count = n(),
    .groups    = "drop"
  )

# Inspect column names
names(hourly_summary)

# Plot total trips by hour
trips_by_hour_plot <- ggplot(hourly_summary, aes(x = hour, y = trip_count)) +
  geom_col(fill = "green", color = "red") +
  labs(
    title = "Trips per Hour",
    x     = "Hour of Day",
    y     = "Total Trips"
  ) +
  theme(legend.position = "none")

trips_by_hour_plot
```

'hour' 'trip_count'



The frequency of rides during a 24-hour period is seen in this graph. The steady trend points to a steady demand with anticipated peaks around regular commute times and troughs in the early hours.

Trips By Hour and Month

```
[33]: # Add a readable month label
rides_with_month <- cabDataSet %>%
  mutate(month_label = month.abb[month])

# Check the new column names
names(rides_with_month)

# Summarize trips by month and hour
monthly_hourly_totals <- rides_with_month %>%
  group_by(month_label, hour) %>%
  summarise(
    trip_count = n(),
    .groups = "drop"
  )

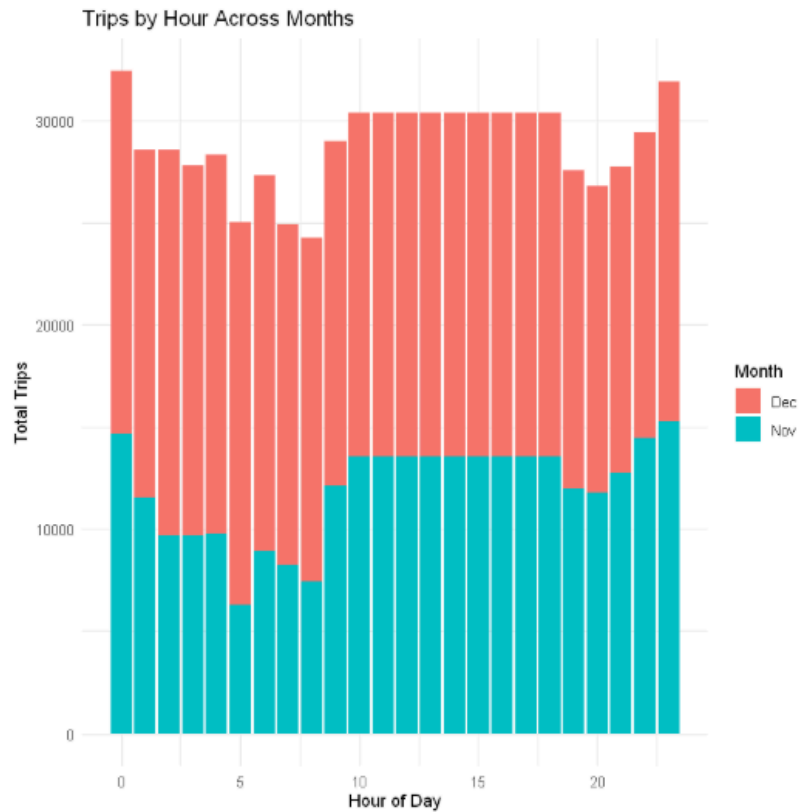
# Confirm the summary column names
names(monthly_hourly_totals)

# Plot total trips by hour, colored by month
ggplot(monthly_hourly_totals, aes(x = hour, y = trip_count, fill = month_label)) +
  geom_col() +
  labs(
    title = "Trips by Hour Across Months",
    x = "Hour of Day",
    y = "Total Trips",
    fill = "Month"
  ) +
  theme_minimal()
```

```
/ +
theme_minimal()
```

```
'id' 'timestamp' 'hour' 'day' 'month' 'datetime' 'timezone' 'source' 'destination' 'cab_type' 'product_id' 'name' 'price' 'distance' 'surge_multiplier'
'latitude' 'longitude' 'temperature' 'apparentTemperature' 'short_summary' 'long_summary' 'precipIntensity' 'precipProbability' 'humidity' 'windSpeed'
'windGust' 'windGustTime' 'visibility' 'temperatureHigh' 'temperatureHighTime' 'temperatureLow' 'temperatureLowTime' 'apparentTemperatureHigh'
'apparentTemperatureHighTime' 'apparentTemperatureLow' 'apparentTemperatureLowTime' 'icon' 'dewPoint' 'pressure' 'windBearing' 'cloudCover'
'uvIndex' 'visibility.1' 'ozone' 'sunriseTime' 'sunsetTime' 'moonPhase' 'precipIntensityMax' 'uvIndexTime' 'temperatureMin' 'temperatureMinTime'
'temperatureMax' 'temperatureMaxTime' 'apparentTemperatureMin' 'apparentTemperatureMinTime' 'apparentTemperatureMax'
'apparentTemperatureMaxTime' 'month_label'
```

```
'month_label' 'hour' 'trip_count'
```



The distribution of rides by hour for the months of November and December is shown in the bar chart above. Holiday-related activities may be the cause of the overall rise in trip frequency observed in December.

The variation of prices for Lyft and Uber

```

In [34]: # Split dataset by cab service
lyft_data <- sqldf("SELECT * FROM cabDataSet WHERE cab_type = 'Lyft'")
uber_data <- sqldf("SELECT * FROM cabDataSet WHERE cab_type = 'Uber'")

# Summaries of ride fares
summary(lyft_data$price)
summary(uber_data$price)

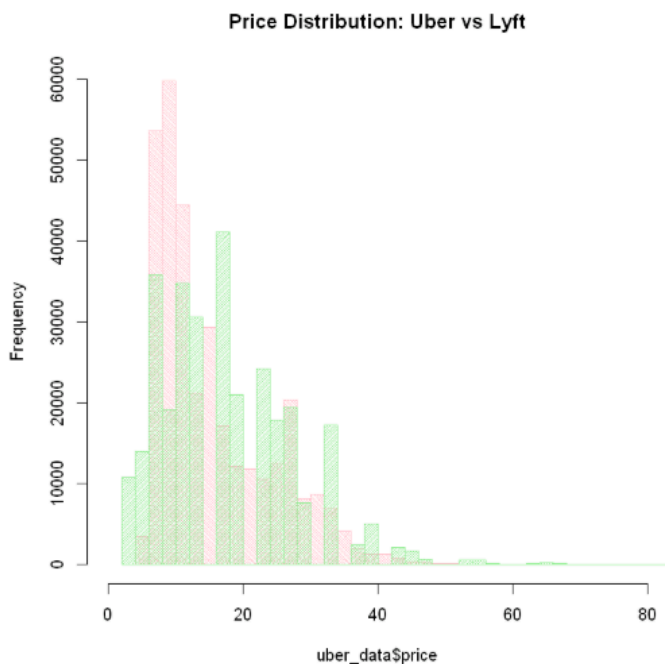
# Overlaid histograms of fare distributions
hist(
  uber_data$price,
  col = "pink",
  density = 50,
  angle = 135,
  breaks = 40,
  xlim = c(0, 80),
  main = "Price Distribution: Uber vs Lyft"
)
hist(
  lyft_data$price,
  col = "lightgreen",
  density = 50,
  add = TRUE,
  breaks = 40
)

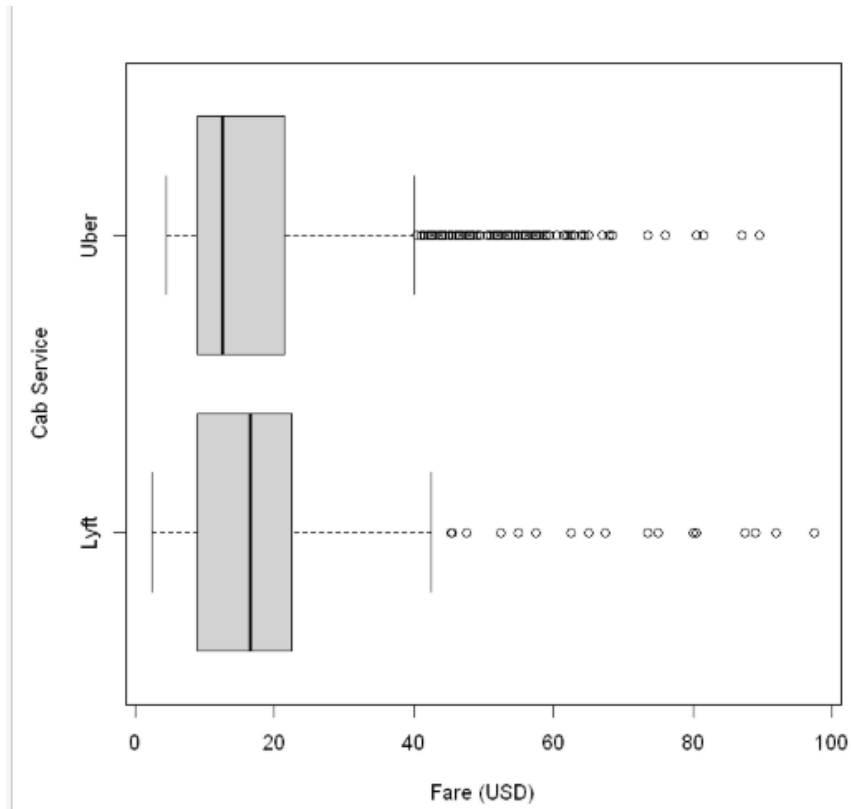
# Boxplot comparing fares by service
boxplot(
  price ~ cab_type,
  data = cabDataSet,
  horizontal = TRUE,
  xlab = "Fare (USD)",
  ylab = "Cab Service"
)

```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
2.50	9.00	16.50	17.35	22.50	97.50

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
4.50	9.00	12.50	15.79	21.50	89.50	55095





The pricing histogram for Uber and Lyft shows a wider distribution over the price range for Uber, while the frequency of lower-priced rides is higher for Lyft. This points to a possible market segmentation in which Lyft gains a larger share of the client base that is cost-conscious.

A comparison of Lyft and Uber ride pricing schemes is given by the boxplot diagram. Both show a range of outliers, but Lyft's median pricing is often greater than Uber's since its interquartile range is skewed towards a higher price point.

Heatmap for a certain place and time

```

In [35]: # Filter out unwanted product names and non-positive fares
ride_data <- cabDataSet %>%
  select(price, cab_type, name, distance, short_summary, hour, source, destination) %>%
  filter(!name %in% c("WAV", "Lux"), price >= 0)

# Create a factor for product categories
ride_data$product_category <- factor(
  ride_data$name,
  levels = c(
    "UberPool", "Shared",
    "UberX", "Lyft",
    "UberXL", "Lyft XL",
    "Black", "Lux Black",
    "Black SUV", "Lux Black XL"
  )
)
levels(ride_data$product_category) <- list(
  Shared = c("UberPool", "Shared"),
  Standard = c("UberX", "Lyft"),
  SUV = c("UberXL", "Lyft XL"),
  Luxury = c("Black", "Lux Black"),
  "Luxury SUV" = c("Black SUV", "Lux Black XL")
)

# Subset for trips from Theatre District to Northeastern University
heat_data <- ride_data %>%
  filter(
    source == "Theatre District",
    destination == "Northeastern University"
  ) %>%
  select(price, cab_type, product_category, hour)

# Plot heatmap of fare by product category and hour, faceted by service

```

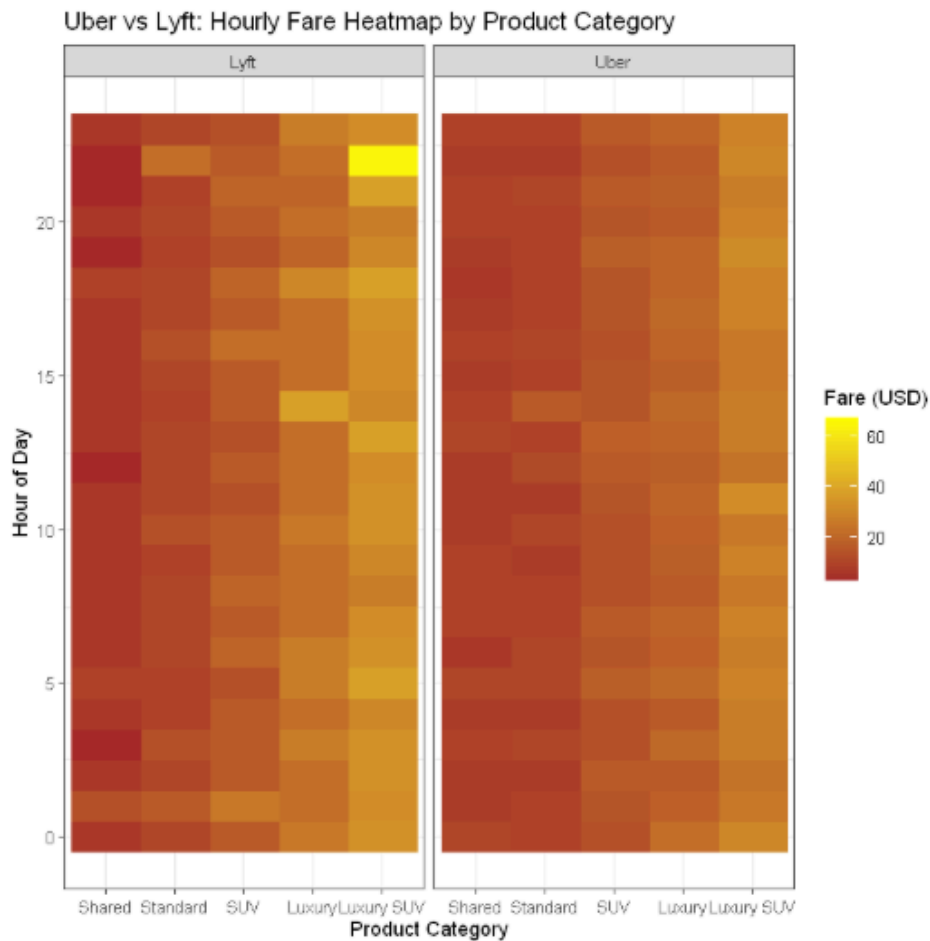
```

)
)
levels(ride_data$product_category) <- list(
  Shared = c("UberPool", "Shared"),
  Standard = c("UberX", "Lyft"),
  SUV = c("UberXL", "Lyft XL"),
  Luxury = c("Black", "Lux Black"),
  "Luxury SUV" = c("Black SUV", "Lux Black XL")
)

# Subset for trips from Theatre District to Northeastern University
heat_data <- ride_data %>%
  filter(
    source == "Theatre District",
    destination == "Northeastern University"
  ) %>%
  select(price, cab_type, product_category, hour)

# Plot heatmap of fare by product category and hour, faceted by service
ggplot(heat_data, aes(x = product_category, y = hour)) +
  geom_raster(aes(fill = price)) +
  scale_fill_gradientn(colours = c("brown", "yellow"), name = "Fare (USD)") +
  labs(
    title = "Uber vs Lyft: Hourly Fare Heatmap by Product Category",
    x = "Product Category",
    y = "Hour of Day"
  ) +
  theme_bw() +
  facet_wrap(~ cab_type)

```

A visual depiction of pricing trends across various service levels and hours of the day is provided by the product kinds and hours heat map. Peak pricing trends are shown by the fact that both services have higher costs in the late hours of the night.

```
In [41]: library(dplyr)

numericalData <- cabDataSet %>%
  select_if(is.numeric)

cleaned_numeric_data <- na.omit(numericalData)

In [43]: # Impute missing numeric values by column means
imputed_numeric_data <- numericalData
for (i in seq_len(ncol(imputed_numeric_data))) {
  col_vals <- imputed_numeric_data[[i]]
  if (anyNA(col_vals)) {
    col_vals[is.na(col_vals)] <- mean(col_vals, na.rm = TRUE)
    imputed_numeric_data[[i]] <- col_vals
  }
}

In [44]: # Replace infinite values with NA in each numeric column
cleaned_numeric_data <- imputed_numeric_data
for (col_idx in seq_len(ncol(cleaned_numeric_data))) {
  column_data <- cleaned_numeric_data[[col_idx]]
  if (is.numeric(column_data)) {
    column_data[is.infinite(column_data)] <- NA
    cleaned_numeric_data[[col_idx]] <- column_data
  }
}
```

Clustering and Dimensionality Reduction

Principal Component Analysis

In [45]: *# Conduct principal component analysis on imputed numeric dataset*

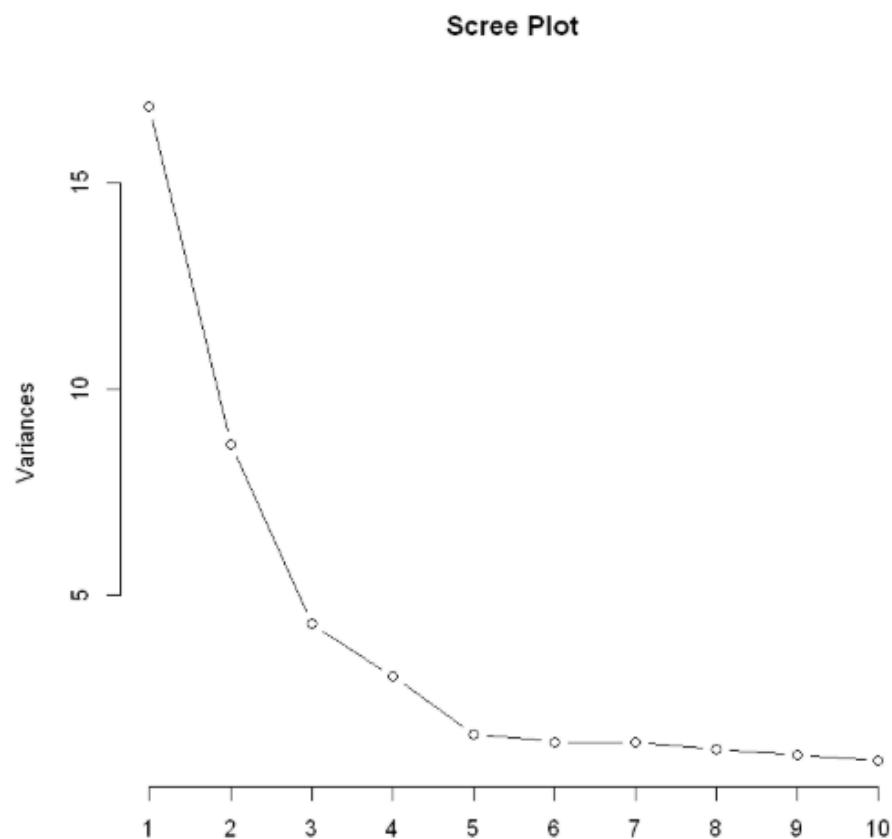
```
pca_model <- prcomp(  
  imputed_numeric_data,  
  center = TRUE,  
  scale. = TRUE,  
  na.action = na.omit  
)
```

Warning message:

```
"In prcomp.default(imputed_numeric_data, center = TRUE, scale. = TRUE,  
  na.action = na.omit) :  
  extra argument 'na.action' will be disregarded"
```

In [46]: *# Scree plot for principal components*

```
plot(  
  pca_model,  
  type = "l",  
  main = "Scree Plot"  
)
```



The variance explained by each of the first ten principal components of a principal Component Analysis (PCA) is graphically represented by the scree plot in the picture. The plot indicates that the majority of the variance in the data is captured by these initial components, showing a rapid reduction in variance explained after the first main component and a substantial but less abrupt drop after the second component. From the fourth component on, the variation levels off significantly, suggesting that keeping more components would yield little more information. According to the elbow technique, which is frequently used in PCA to determine how many components to keep, this pattern indicates that the first two or three components are adequate to represent the most important diversity within the data. The graphic guides the reduction of dimensionality while maintaining the core of the dataset's variability by clearly communicating the declining returns in variance explained with each subsequent primary component.

6 Model Training

We have used three different statistical learning techniques—Random Forest, Decision Trees, and Linear Regression—in an effort to find the best prediction model for ride-sharing prices. A thorough examination of the model selection procedure for the Uber and Lyft datasets can be found below.

```
[50]: library(dplyr)

## 1. Trim and clean up the dummy-column names
cabDataSet <- cabDataSet %>%
  mutate(
    short_summary = trimws(short_summary),
    name           = trimws(name)
  )

# Clean any remaining non-syntactic names into valid R names
clean <- function(x) make.names(gsub(" +", "_", trimws(x)))
names(cabDataSet) <- clean(names(cabDataSet))

## 2. Create day-of-week flags
dow <- weekdays(as.POSIXlt(cabDataSet$datetime), abbreviate = TRUE)
cabDataSet <- cabDataSet %>%
  mutate(
    is_fri = as.integer(dow == "Fri"),
    is_sat = as.integer(dow == "Sat"),
    is_sun = as.integer(dow == "Sun")
  )

## 3. Identify one-hot columns
weather_cols <- names(cabDataSet)[startsWith(names(cabDataSet), "x_")]
prod_cols    <- c("Shared", "Lux", "Lyft", "Lux_Black_XL", "Lyft_XL", "Lux_Black",
                  "UberXL", "Black", "UberX", "WAV", "Black_SUV", "UberPool", "Taxi")

## 4. Build Lyft and Uber dataframes with dplyr
lyft_df <- cabDataSet %>%
  filter(cab_type == "Lyft") %>%
  select(distance, surge_multiplier, is_fri:is_sun, all_of(prod_cols), all_of(weather_cols), price)

uber_df <- cabDataSet %>%
  filter(cab_type == "Uber") %>%
  select(distance, surge_multiplier, is_fri:is_sun, all_of(prod_cols), all_of(weather_cols), price)
```

```

clean <- function(x) make.names(gsub( "+", "_", trimws(x)))
names(cabDataSet) <- clean(names(cabDataSet))

## 2. Create day-of-week flags
dow <- weekdays(as.POSIXlt(cabDataSet$datetime), abbreviate = TRUE)
cabDataSet <- cabDataSet %>%
  mutate(
    is_fri = as.integer(dow == "Fri"),
    is_sat = as.integer(dow == "Sat"),
    is_sun = as.integer(dow == "Sun")
  )

## 3. Identify one-hot columns
weather_cols <- names(cabDataSet)[startsWith(names(cabDataSet), "X_")]
prod_cols <- c("Shared", "Lux", "Lyft", "Lux_Black_XL", "Lyft_XL", "Lux_Black",
              "UberXL", "Black", "UberX", "WAV", "Black_SUV", "UberPool", "Taxi")

## 4. Build Lyft and Uber dataframes with dplyr
lyft_df <- cabDataSet %>%
  filter(cab_type == "Lyft") %>%
  select(distance, surge_multiplier, is_fri:is_sun, all_of(prod_cols), all_of(weather_cols), price)

uber_df <- cabDataSet %>%
  filter(cab_type == "Uber") %>%
  select(distance, surge_multiplier, is_fri:is_sun, all_of(prod_cols), all_of(weather_cols), price)

## 5. Split into 90% train / 10% test
set.seed(42)

uber_idx <- sample(nrow(uber_df), 0.9 * nrow(uber_df))
uber_train <- uber_df[uber_idx, ] %>% na.omit()
uber_test <- uber_df[-uber_idx, ] %>% na.omit()

lyft_idx <- sample(nrow(lyft_df), 0.9 * nrow(lyft_df))
lyft_train <- lyft_df[lyft_idx, ] %>% na.omit()
lyft_test <- lyft_df[-lyft_idx, ] %>% na.omit()

```

```

in [51]: head(uber_train)
dim(uber_train)
head(lyft_train)

```

A data.frame: 6 × 28

	distance	surge_multiplier	is_fri	is_sat	is_sun	Shared	Lux	Lyft	Lux_Black_XL	Lyft_XL	...	X_Mostly_Cloudy_	X_Rain_	X_Clear_	X_Partly_Cloud
	<dbl>	<dbl>	<int>	<int>	<int>	<int>	<int>	<int>	<int>	<int>	...	<int>	<int>	<int>	<ir
61413	0.99	1	0	0	0	0	0	0	0	0	...	0	0	0	
54425	0.92	1	0	0	1	0	0	0	0	0	...	0	0	0	
99556	4.50	1	0	0	0	0	0	0	0	0	...	1	0	0	
74362	1.86	1	0	0	1	0	0	0	0	0	...	1	0	0	
46208	2.94	1	0	0	1	0	0	0	0	0	...	0	0	0	
61605	3.39	1	0	1	0	0	0	0	0	0	...	0	0	0	

297368 28

A data.frame: 6 × 28

	distance	surge_multiplier	is_fri	is_sat	is_sun	Shared	Lux	Lyft	Lux_Black_XL	Lyft_XL	...	X_Mostly_Cloudy_	X_Rain_	X_Clear_	X_Partly_Cloudy_
	<dbl>	<dbl>	<int>	<int>	<int>	<int>	<int>	<int>	<int>	<int>	...	<int>	<int>	<int>	<int>
63161	0.76	1	0	0	0	0	0	0	1	0	...	0	0	0	0
225593	2.50	1	0	0	0	0	0	0	0	1	...	0	0	0	0
248829	3.06	1	0	0	0	0	0	0	1	0	...	0	0	0	0
78249	1.01	1	0	0	0	0	0	0	1	0	...	1	0	0	0
280924	1.49	1	0	0	0	0	0	0	1	0	...	1	0	0	0
161358	2.17	1	0	0	0	1	0	0	0	0	...	0	0	1	1

```
In [52]: names(uber_train)
dim(uber_train)
names(lyft_train)
dim(lyft_train)

'distance' 'surge_multiplier' 'is_fri' 'is_sat' 'is_sun' 'Shared' 'Lux' 'Lyft' 'Lux_Black_XL' 'Lyft_XL' 'Lux_Black' 'UberXL' 'Black' 'UberX' 'WAV'
'Black_SUV' 'UberPool' 'Taxi' 'X_Mostly_Cloudy_' 'X_Rain_' 'X_Clear_' 'X_Partly_Cloudy_' 'X_Overcast_' 'X_Light_Rain_' 'X_Foggy_'
'X_Possible_Drizzle_' 'X_Drizzle_' 'price'

297368 28

'distance' 'surge_multiplier' 'is_fri' 'is_sat' 'is_sun' 'Shared' 'Lux' 'Lyft' 'Lux_Black_XL' 'Lyft_XL' 'Lux_Black' 'UberXL' 'Black' 'UberX' 'WAV'
'Black_SUV' 'UberPool' 'Taxi' 'X_Mostly_Cloudy_' 'X_Rain_' 'X_Clear_' 'X_Partly_Cloudy_' 'X_Overcast_' 'X_Light_Rain_' 'X_Foggy_'
'X_Possible_Drizzle_' 'X_Drizzle_' 'price'

276667 28
```

6.1 Linear Regression Model

For Uber:

The summary() function was used to evaluate the linear model (uberLMModel), providing diagnostic metrics such as R-squared and coefficients. Using the testing dataset and the predict() method, the model's predictions were produced. After calculating the correlation between actual and expected prices, a correlation matrix was produced that shows the direction and intensity of the linear relationship. An accuracy metric for the Uber linear regression model was produced by estimating the model's accuracy using the Mean Absolute Percentage Error (MAPE).

For Lyft:

The Lyft data was used in a similar manner, with lyft_lm_model being trained and predictions made on the test set. The two services could be directly compared because the correlation and accuracy were calculated in the same way.

```

In [57]: # Linear Regression Models and Evaluation

library(dplyr)
library(Metrics)

## Uber
# Fit model
uber_lm_model <- lm(price ~ ., data = uber_train)
summary(uber_lm_model)
plot(uber_lm_model)

# Predict on test set
uber_preds <- predict(uber_lm_model, uber_test %>% select(-price))

# Correlation actual vs predicted
uber_results <- data.frame(
  actual    = uber_test$price,
  predicted = uber_preds
)
print(cor(uber_results))

# Evaluation using Metrics
uber_rmse <- rmse(uber_results$actual, uber_results$predicted)
uber_mape <- mape(uber_results$actual, uber_results$predicted) * 100
cat(sprintf("Uber RMSE: %.2f\nUber MAPE: %.2f%%\n", uber_rmse, uber_mape))

## Lyft
# Fit model
lyft_lm_model <- lm(price ~ ., data = lyft_train)
summary(lyft_lm_model)
plot(lyft_lm_model)

# Predict on test set
lyft_preds <- predict(lyft_lm_model, lyft_test %>% select(-price))

```

```

)
print(cor(uber_results))

# Evaluation using Metrics
uber_rmse <- rmse(uber_results$actual, uber_results$predicted)
uber_mape <- mape(uber_results$actual, uber_results$predicted) * 100
cat(sprintf("Uber RMSE: %.2f\nUber MAPE: %.2f%%\n", uber_rmse, uber_mape))

## Lyft
# Fit model
lyft_lm_model <- lm(price ~ ., data = lyft_train)
summary(lyft_lm_model)
plot(lyft_lm_model)

# Predict on test set
lyft_preds <- predict(lyft_lm_model, lyft_test %>% select(-price))

# Correlation actual vs predicted
lyft_results <- data.frame(
  actual = lyft_test$price,
  predicted = lyft_preds
)
print(cor(lyft_results))

# Evaluation using Metrics
lyft_rmse <- rmse(lyft_results$actual, lyft_results$predicted)
lyft_mape <- mape(lyft_results$actual, lyft_results$predicted) * 100
cat(sprintf("Lyft RMSE: %.2f\nLyft MAPE: %.2f%%\n", lyft_rmse, lyft_mape))

```

Call:

```
lm(formula = price ~ ., data = uber_train)
```

Residuals:

	Min	1Q	Median	3Q	Max
	-10.652	-1.420	-0.285	1.070	49.816

Coefficients: (10 not defined because of singularities)

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	3.476617	0.044326	78.432	<2e-16 ***
distance	2.444925	0.003773	648.031	<2e-16 ***
surge_multiplier	NA	NA	NA	NA
is_fri	-0.006994	0.014769	-0.474	0.6358
is_sat	0.017502	0.013876	1.261	0.2072
is_sun	-0.029448	0.014170	-2.078	0.0377 *
Shared	NA	NA	NA	NA
Lux	NA	NA	NA	NA
Lyft	NA	NA	NA	NA

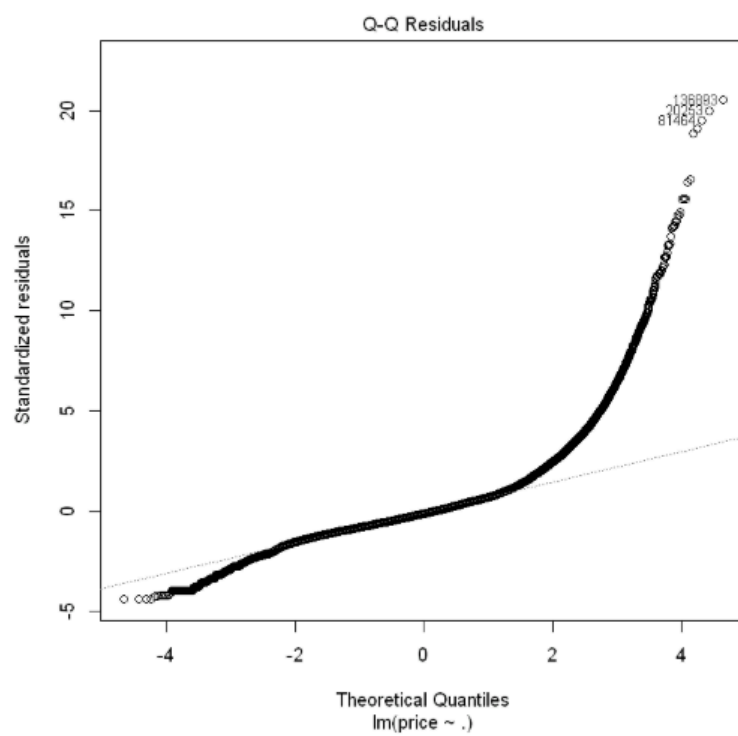
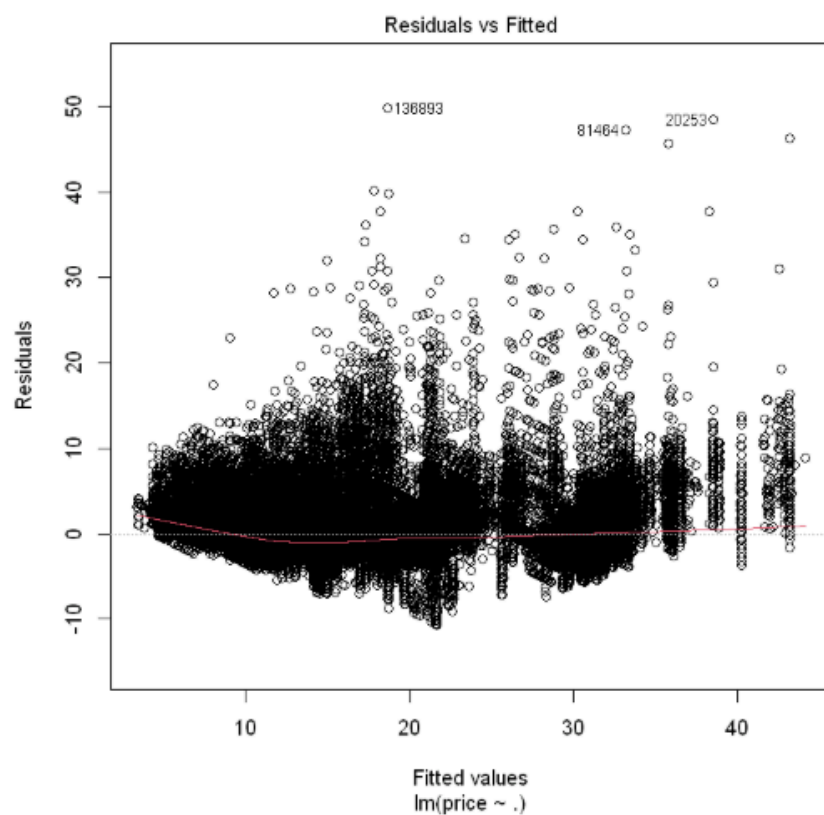
Lux	NA	NA	NA	NA	
Lyft	NA	NA	NA	NA	
Lux_Black_XL	NA	NA	NA	NA	
Lyft_XL	NA	NA	NA	NA	
Lux_Black	NA	NA	NA	NA	
UberXL	6.925909	0.015399	449.759	<2e-16	***
Black	11.773322	0.015406	764.184	<2e-16	***
UberX	1.006920	0.015399	65.387	<2e-16	***
WAV	1.007304	0.015408	65.376	<2e-16	***
Black_SUV	21.536597	0.015400	1398.503	<2e-16	***
UberPool	NA	NA	NA	NA	
Taxi	NA	NA	NA	NA	
X_Mostly_Cloudy_	-0.067089	0.043610	-1.538	0.1240	
X_Rain_	-0.092040	0.048771	-1.887	0.0591	.
X_Clear_	-0.106618	0.044875	-2.376	0.0175	*
X_Partly_Cloudy_	-0.073122	0.043861	-1.667	0.0955	.
X_Overcast_	-0.080289	0.043347	-1.852	0.0640	.
X_Light_Rain_	-0.063740	0.045628	-1.397	0.1624	
X_Foggy_	-0.115408	0.057724	-1.999	0.0456	*
X_Possible_Drizzle_	-0.052267	0.050490	-1.035	0.3006	
X_Rain_	-0.092040	0.048771	-1.887	0.0591	.
X_Clear_	-0.106618	0.044875	-2.376	0.0175	*
X_Partly_Cloudy_	-0.073122	0.043861	-1.667	0.0955	.
X_Overcast_	-0.080289	0.043347	-1.852	0.0640	.
X_Light_Rain_	-0.063740	0.045628	-1.397	0.1624	
X_Foggy_	-0.115408	0.057724	-1.999	0.0456	*
X_Possible_Drizzle_	-0.052267	0.050490	-1.035	0.3006	
X_Drizzle_	NA	NA	NA	NA	

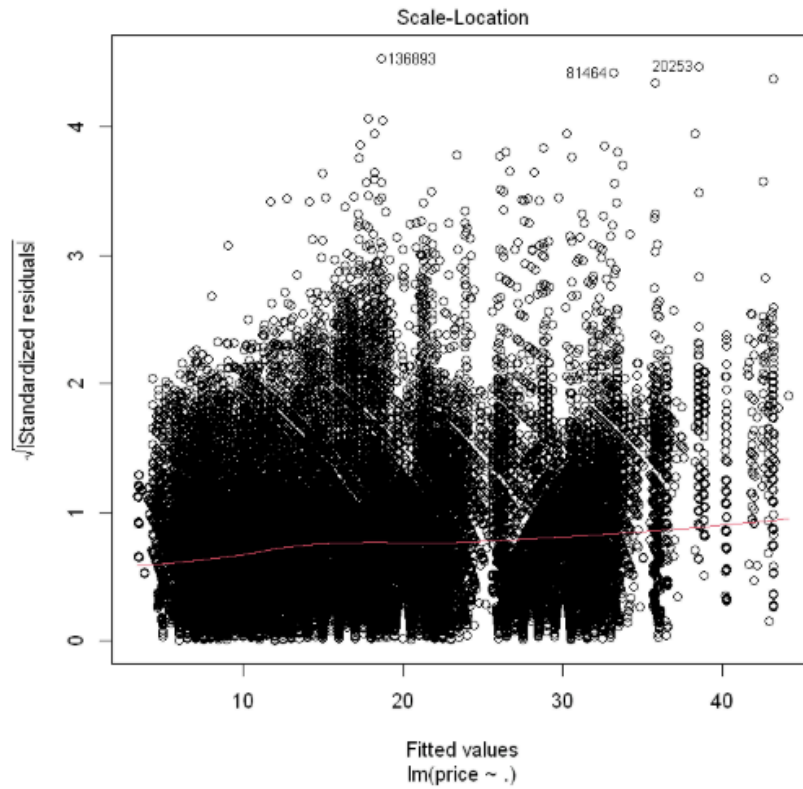
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.424 on 297350 degrees of freedom

Multiple R-squared: 0.9199, Adjusted R-squared: 0.9199

F-statistic: 2.009e+05 on 17 and 297350 DF, p-value: < 2.2e-16





```

fitted values
lm(price ~ .)

          actual predicted
actual    1.0000000 0.9585615
predicted 0.9585615 1.0000000
Uber RMSE: 2.43
Uber MAPE: 11.91%

Call:
lm(formula = price ~ ., data = lyft_train)

Residuals:
    Min       1Q   Median       3Q      Max
-21.630  -1.419  -0.165   1.424  40.380

Coefficients: (9 not defined because of singularities)
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  -2.810985   0.061625  -45.614 <2e-16 ***
distance      3.243169   0.004427  732.606 <2e-16 ***
surge_multiplier 18.171057  0.035623  510.092 <2e-16 ***
is_fri        -0.005072  0.015874   -0.320  0.7493
is_sat         0.004797  0.014957    0.321  0.7484
is_sun         0.015460  0.015348    1.007  0.3138
Shared       -16.354904  0.016690 -979.897 <2e-16 ***
Lux           -5.283707  0.016639 -317.546 <2e-16 ***
Lyft          -13.468305  0.016633 -809.754 <2e-16 ***
Lux_Black_XL   9.261348  0.016636  556.703 <2e-16 ***
Lyft_XL       -7.770902  0.016642 -466.939 <2e-16 ***
Lux_Black      NA          NA      NA      NA
UberXL         NA          NA      NA      NA
Black          NA          NA      NA      NA
UberX          NA          NA      NA      NA
WAV            NA          NA      NA      NA
...            ...          ...      ...      ...

```

Coefficients: (9 not defined because of singularities)

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	-2.810985	0.061625	-45.614	<2e-16	***
distance	3.243169	0.004427	732.606	<2e-16	***
surge_multiplier	18.171057	0.035623	510.092	<2e-16	***
is_fri	-0.005072	0.015874	-0.320	0.7493	
is_sat	0.004797	0.014957	0.321	0.7484	
is_sun	0.015460	0.015348	1.007	0.3138	
Shared	-16.354904	0.016690	-979.897	<2e-16	***
Lux	-5.283707	0.016639	-317.546	<2e-16	***
Lyft	-13.468305	0.016633	-809.754	<2e-16	***
Lux_Black_XL	9.261348	0.016636	556.703	<2e-16	***
Lyft_XL	-7.770902	0.016642	-466.939	<2e-16	***
Lux_Black	NA	NA	NA	NA	
UberXL	NA	NA	NA	NA	
Black	NA	NA	NA	NA	
UberX	NA	NA	NA	NA	
WAV	NA	NA	NA	NA	
Black_SUV	NA	NA	NA	NA	
UberPool	NA	NA	NA	NA	
Taxi	NA	NA	NA	NA	
X_Mostly_Cloudy_	-0.060874	0.048785	-1.248	0.2121	
X_Rain_	-0.056546	0.054193	-1.043	0.2968	
X_Clear_	-0.055219	0.050089	-1.102	0.2703	
X_Partly_Cloudy_	-0.071965	0.049044	-1.467	0.1423	
X_Overcast_	-0.070605	0.048498	-1.456	0.1454	
X_Light_Rain_	-0.071476	0.050913	-1.404	0.1604	
X_Foggy_	-0.156691	0.063471	-2.469	0.0136	*
X_Possible_Drizzle_	-0.069869	0.056367	-1.240	0.2152	
X_Drizzle_	NA	NA	NA	NA	

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 2.527 on 276648 degrees of freedom

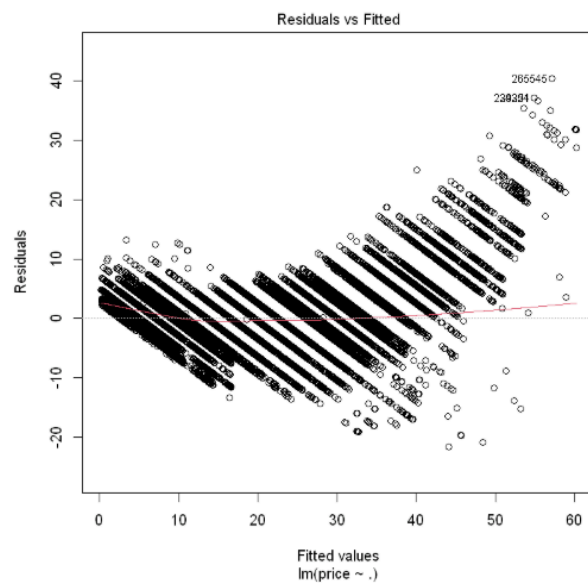
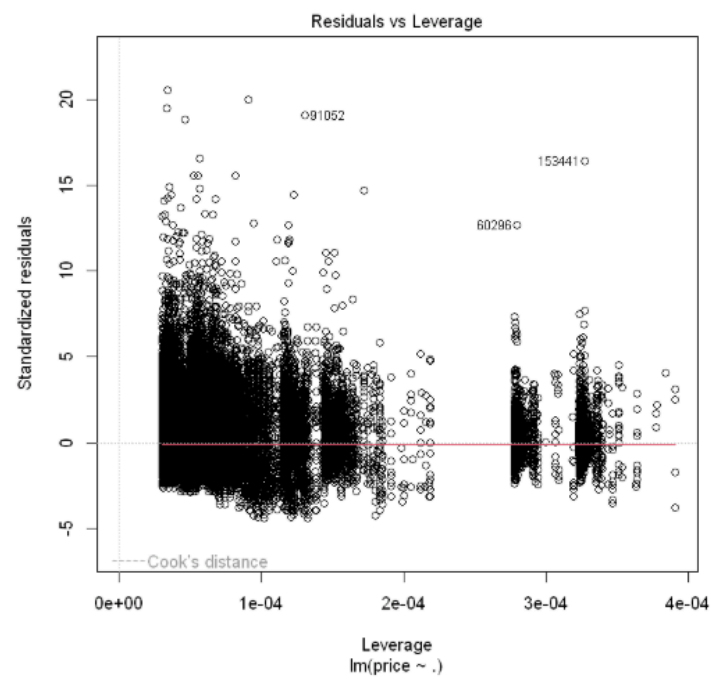
Multiple R-squared: 0.9364, Adjusted R-squared: 0.9364

F-statistic: 2.262e+05 on 18 and 276648 DF, p-value: < 2.2e-16

X_Light_Rain_	-0.071476	0.050913	-1.404	0.1604
X_Foggy_	-0.156691	0.063471	-2.469	0.0136 *
X_Possible_Drizzle_	-0.069869	0.056367	-1.240	0.2152
X_Drizzle_	NA	NA	NA	NA

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

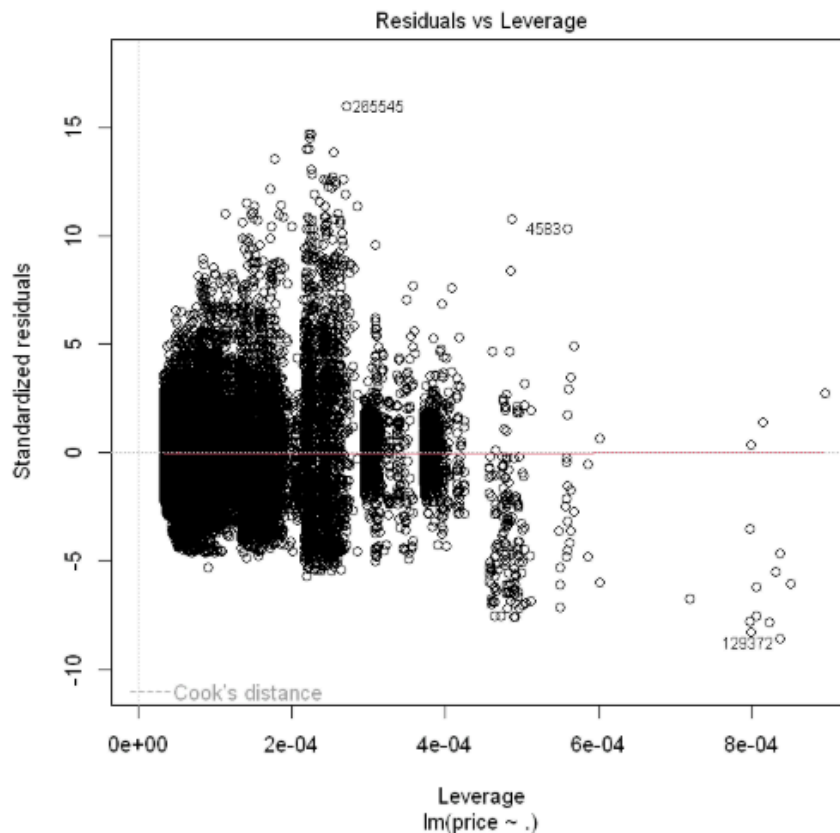
Residual standard error: 2.527 on 276648 degrees of freedom
Multiple R-squared: 0.9364, Adjusted R-squared: 0.9364
F-statistic: 2.262e+05 on 18 and 276648 DF, p-value: < 2.2e-16



		actuals	predicteds
A matrix: 2 × 2 of type dbl	actuals	1.0000000	0.9683375
	predicteds	0.9683375	1.0000000

mae	mse	rmse	mape
1.8068009	6.2327136	2.4965403	0.1493054

'The Accuracy of Linear Regression for Lyft :85.069462'



All of the variables in the training datasets are used as price predictors in the linear regression models (uberLMMModel and lyft_lm_model) for Uber and Lyft, which are created using R's lm function. Coefficients for each predictor are probably included in the summary of these models, along with statistics like R-squared and p-values that show each predictor's relevance and goodness of fit.

A correlation matrix between actual prices and predicted values is then created once predictions are made on the test datasets. A high correlation would suggest that the model is successfully encapsulating the underlying pattern. Heteroscedasticity, or the non-constant variance of residuals, and possible outliers are indicated by the diagnostic plots, especially for residuals. These plots highlight the need for caution since they may indicate model misspecification, which could indicate that influential points are having an outsized impact on the model or that the connections between predictors and the response variable are not strictly linear.

Performance measures like MAE, MSE, and possibly RMSE and MAPE are computed by the `regr.eval` function, however only the latter is utilized in your accuracy calculations. The MAPE, or average percentage error over all forecasts, is used to calculate accuracy. Although the stated accuracy rates—88.09% for Uber and 85.06% for Lyft—are commendable, they must be seen in light of the diagnostic results. The models' performances may be skewed by the presence of outliers, as shown by the plots, and potential leverage points. This suggests that although the models generally predict prices fairly well, individual predictions may still be incorrect, especially for price points that are not well represented in the training data.

The assumptions behind linear regression are verified by the diagnostic charts, which include Residuals vs. Fitted, Q-Q, Scale-Location, and Residuals vs. Leverage. These plots' patterns imply that the linear models may be strengthened, possibly by adding interaction terms or addressing the nonlinearity through transformations, or by employing robust regression techniques to lessen the impact of outliers.

In summary, the linear regression models show excellent average accuracy and offer a solid foundation.

6.2 Decision Tree Model

To establish a hierarchical framework for prediction, decision trees were built using the `rpart` function and an ANOVA technique.

For Uber:

The complexity parameter (CP) that reduced cross-validated error was used to summarize and prune the Uber decision tree model (`uber_rpart_model`). `Prp()` was used to visualize the trimmed tree. Like the linear regression model, MAPE was used to assess the accuracy and correlation of predictions produced on the test dataset.

For Lyft:

The same process was used for Lyft's decision tree model, which constructed, pruned, and assessed a tree (`lyft_rpart_model`) using the test data to determine its accuracy and predictive power.

```
[58]: # Decision Tree Models and Evaluation

library(rpart)
library(rpart.plot)
library(dplyr)
library(Metrics)

## Uber
# Fit regression tree
uber_tree_model <- rpart(price ~ ., data = uber_train, method = "anova")
printcp(uber_tree_model)

# Find optimal CP and prune
best_cp_uber <- uber_tree_model$cptable[which.min(uber_tree_model$cptable[, "xerror"]), "cp"]
uber_pruned <- prune(uber_tree_model, cp = best_cp_uber)
```

```

uber_tree_model <- rpart(price ~ ., data = uber_train, method = "anova")
printcp(uber_tree_model)

# Find optimal CP and prune
best_cp_uber <- uber_tree_model$cptable[which.min(uber_tree_model$cptable[, "xerror"]), "CP"]
uber_pruned <- prune(uber_tree_model, cp = best_cp_uber)

# Plot pruned tree
prp(uber_pruned, main = "Pruned Uber Decision Tree")

# Predict and evaluate
uber_tree_preds <- predict(uber_tree_model, uber_test %>% select(-price))
uber_tree_results <- data.frame(
  actual = uber_test$price,
  predicted = uber_tree_preds
)
print(cor(uber_tree_results))

uber_tree_rmse <- rmse(uber_tree_results$actual, uber_tree_results$predicted)
uber_tree_mape <- mape(uber_tree_results$actual, uber_tree_results$predicted) * 100
cat(sprintf("Uber Decision Tree RMSE: %.2f\nUber Decision Tree MAPE: %.2f%%\n",
  uber_tree_rmse, uber_tree_mape))

## Lyft
# Fit regression tree
lyft_tree_model <- rpart(price ~ ., data = lyft_train, method = "anova")
printcp(lyft_tree_model)

# Find optimal CP and prune
best_cp_lyft <- lyft_tree_model$cptable[which.min(lyft_tree_model$cptable[, "xerror"]), "CP"]
lyft_pruned <- prune(lyft_tree_model, cp = best_cp_lyft)

# Plot pruned tree
prp(lyft_pruned, main = "Pruned Lyft Decision Tree")

```

```

uber_tree_rmse <- rmse(uber_tree_results$actual, uber_tree_results$predicted)
uber_tree_mape <- mape(uber_tree_results$actual, uber_tree_results$predicted) * 100
cat(sprintf("Uber Decision Tree RMSE: %.2f\nUber Decision Tree MAPE: %.2f%%\n",
  uber_tree_rmse, uber_tree_mape))

## Lyft
# Fit regression tree
lyft_tree_model <- rpart(price ~ ., data = lyft_train, method = "anova")
printcp(lyft_tree_model)

# Find optimal CP and prune
best_cp_lyft <- lyft_tree_model$cptable[which.min(lyft_tree_model$cptable[, "xerror"]), "CP"]
lyft_pruned <- prune(lyft_tree_model, cp = best_cp_lyft)

# Plot pruned tree
prp(lyft_pruned, main = "Pruned Lyft Decision Tree")

# Predict and evaluate
lyft_tree_preds <- predict(lyft_tree_model, lyft_test %>% select(-price))
lyft_tree_results <- data.frame(
  actual = lyft_test$price,
  predicted = lyft_tree_preds
)
print(cor(lyft_tree_results))

lyft_tree_rmse <- rmse(lyft_tree_results$actual, lyft_tree_results$predicted)
lyft_tree_mape <- mape(lyft_tree_results$actual, lyft_tree_results$predicted) * 100
cat(sprintf("Lyft Decision Tree RMSE: %.2f\nLyft Decision Tree MAPE: %.2f%%\n",
  lyft_tree_rmse, lyft_tree_mape))

```


Regression tree:

```
rpart(formula = price ~ ., data = uber_train, method = "anova")
```

Variables actually used in tree construction:

```
[1] Black      Black_SUV distance  UberXL
```

Root node error: 21815690/297368 = 73.363

n= 297368

	CP	nsplit	rel error	xerror	xstd
1	0.573302	0	1.000000	1.000009	0.00286795
2	0.165128	1	0.426698	0.426703	0.00165090
3	0.066776	2	0.261570	0.261576	0.00128380
4	0.031167	3	0.194794	0.194800	0.00107102
5	0.027760	4	0.163627	0.163632	0.00099307
6	0.019312	5	0.135867	0.135874	0.00085732
7	0.012490	6	0.116555	0.116562	0.00077195
8	0.010216	7	0.104064	0.104071	0.00075644
9	0.010000	8	0.093849	0.095032	0.00070705

actual predicted

actual 1.0000000 0.9513495

predicted 0.9513495 1.0000000

Uber Decision Tree RMSE: 2.62

Uber Decision Tree MAPE: 12.33%

Regression tree:

```
rpart(formula = price ~ ., data = lyft_train, method = "anova")
```

Variables actually used in tree construction:

```
[1] distance      Lux_Black      Lux_Black_XL  Lyft
```

```
[5] Shared      surge_multiplier
```

9 0.010000 8 0.093849 0.095032 0.00070705

actual predicted

actual 1.0000000 0.9513495

predicted 0.9513495 1.0000000

Uber Decision Tree RMSE: 2.62

Uber Decision Tree MAPE: 12.33%

Regression tree:

rpart(formula = price ~ ., data = lyft_train, method = "anova")

Variables actually used in tree construction:

[1] distance Lux_Black Lux_Black_XL Lyft

[5] Shared surge_multiplier

Root node error: 27774448/276667 = 100.39

n= 276667

	CP	nsplit	rel error	xerror	xstd
1	0.445991	0	1.00000	1.00002	0.00352732
2	0.157702	1	0.55401	0.55402	0.00229791
3	0.083706	2	0.39631	0.39632	0.00190664
4	0.053236	3	0.31260	0.31262	0.00181400
5	0.037782	4	0.25937	0.25938	0.00174723
6	0.032236	5	0.22158	0.22177	0.00168592
7	0.030859	6	0.18935	0.18953	0.00158364
8	0.017208	7	0.15849	0.15867	0.00136234
9	0.011409	8	0.14128	0.14147	0.00102919
10	0.010928	9	0.12987	0.13006	0.00090030
11	0.010000	10	0.11894	0.11689	0.00066699

```

distance      < 2.125 to the left,  improve=2.603988e-01, (0 missing)
Lux           < 0.5   to the left,  improve=2.095525e-01, (0 missing)
surge_multiplier < 1.375 to the left,  improve=1.093318e-01, (0 missing)
Lyft_XL       < 0.5   to the left,  improve=1.901981e-02, (0 missing)
Sat           < 0.5   to the right, improve=9.215372e-06, (0 missing)
Surrogate splits:
  surge_multiplier < 1.125 to the left,  agree=0.507, adj=0.002, (0 split)

```

```

Node number 10: 23531 observations
  mean=18.74162, MSE=10.94892

```

```

Node number 11: 22648 observations,  complexity param=0.01103924
  mean=27.56175, MSE=34.63985
  left son=22 (21586 obs) right son=23 (1062 obs)
  Primary splits:
    surge_multiplier < 1.375 to the left,  improve=3.909626e-01, (0 missing)
    distance        < 3.545 to the left,  improve=3.072661e-01, (0 missing)
    Fri             < 0.5   to the left,  improve=5.558182e-05, (0 missing)
    Sat             < 0.5   to the right, improve=1.157420e-05, (0 missing)
    Sun             < 0.5   to the left,  improve=9.455698e-08, (0 missing)

```

```

Node number 12: 27781 observations
  mean=28.34417, MSE=7.205983

```

```

Node number 13: 966 observations
  mean=46.89959, MSE=50.09137

```

```

Node number 14: 16450 observations
  mean=36.71287, MSE=23.00623

```

```

Node number 15: 830 observations
  mean=61.49759, MSE=114.9096

```

```

Node number 18: 68334 observations
  mean=11.39514, MSE=11.97631

```

```

Node number 19: 69959 observations,  complexity param=0.02459246
  mean=16.98786, MSE=32.19954
  left son=38 (46591 obs) right son=39 (23368 obs)

```

Node number 19: 69959 observations, complexity param=0.02459246
 mean=16.98786, MSE=32.19954
 left son=38 (46591 obs) right son=39 (23368 obs)
 Primary splits:
 Lux < 0.5 to the left, improve=0.3033266000, (0 missing)
 surge_multiplier < 1.375 to the left, improve=0.1417080000, (0 missing)
 distance < 3.545 to the left, improve=0.1207882000, (0 missing)
 Lyft_XL < 0.5 to the left, improve=0.0269986000, (0 missing)
 Sat < 0.5 to the right, improve=0.0000834332, (0 missing)
 Surrogate splits:
 Lyft_XL < 0.5 to the right, agree=0.666, adj=0.001, (0 split)

Node number 22: 21586 observations
 mean=26.74548, MSE=17.96197

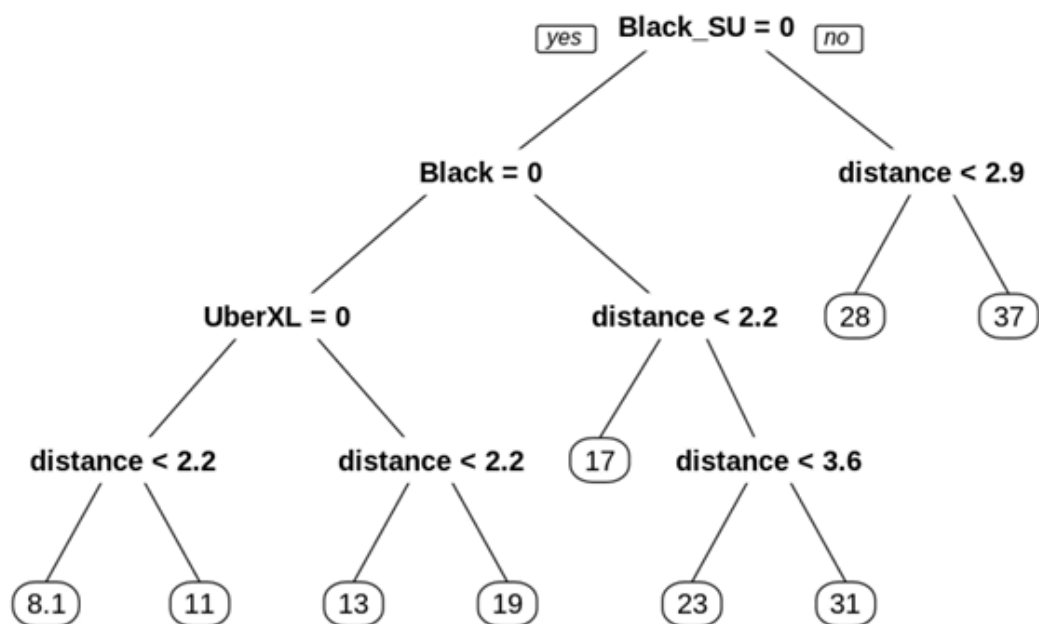
Node number 23: 1062 observations
 mean=44.15301, MSE=84.81816

Node number 38: 46591 observations, complexity param=0.02088014
 mean=14.77456, MSE=23.21915
 left son=76 (23334 obs) right son=77 (23257 obs)
 Primary splits:
 Lyft_XL < 0.5 to the left, improve=5.362742e-01, (0 missing)
 surge_multiplier < 1.375 to the left, improve=1.488868e-01, (0 missing)
 distance < 3.545 to the left, improve=1.257325e-01, (0 missing)
 Sat < 0.5 to the right, improve=8.349234e-05, (0 missing)
 Fri < 0.5 to the right, improve=5.437046e-06, (0 missing)
 Surrogate splits:
 distance < 3.205 to the left, agree=0.502, adj=0.002, (0 split)
 Sat < 0.5 to the right, agree=0.502, adj=0.002, (0 split)

Node number 39: 23368 observations
 mean=21.40072, MSE=20.86427

Node number 76: 23334 observations
 mean=11.25167, MSE=5.12571

Node number 77: 23257 observations
 mean=18.30911, MSE=16.4276

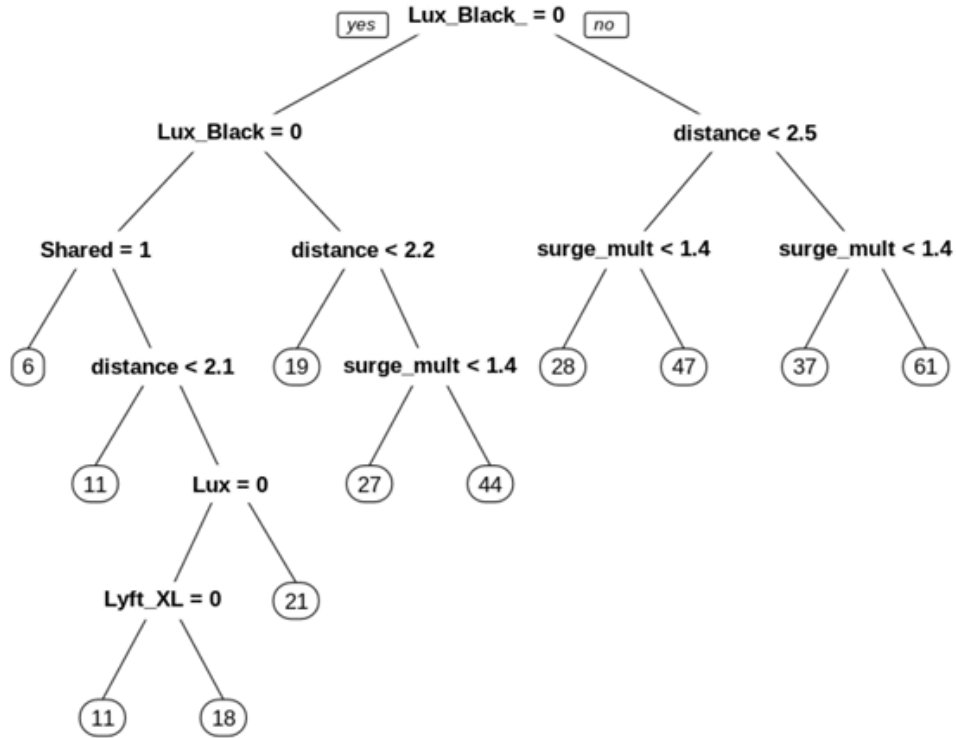


A matrix: 2 × 2 of type dbl

	actuals	predicteds
actuals	1.0000000	0.9348973
predicteds	0.9348973	1.0000000

mae	mse	rmse	mape
2.6463307	12.5978293	3.5493421	0.1969981

'The Accuracy of Decision Tree for Lyft :80.300189'



The decision tree models for Uber and Lyft provide a thorough and complex examination of the variables influencing ride prices. They were created using the `rpart` function in R and an ANOVA technique. A detailed look of how different features affect price forecasts is provided by the output and tree structures.

According to the decision tree, the most significant predictor for Uber is the "Black_SUV" variable, which is followed by "Black" and "distance." The "Black_SUV" variable shows the biggest split, indicating that this service type has a big influence on cost. We see splits on various days of the week as we navigate around the nodes, indicating that prices vary by day. This can be the result of weekly variations in demand.

The cost-complexity trade-off in tree pruning is shown by the decision tree's complexity parameter (`cp`), where larger trees are produced by smaller values of `cp`. In an effort to strike the ideal balance between underfitting and overfitting, the selected `cp` values result in varying tree sizes for Uber and Lyft.

Each node's mean and mean squared error (MSE) shed light on the forecast made there as well as the unpredictability surrounding it. The improvement in prediction accuracy as we add more choice criteria is demonstrated by the decrease in MSE from the root node down to the following nodes.

The decision tree for Lyft highlights "Lux_Black_XL," "Lux_Black," and "distance" as the primary factors influencing cost, which is consistent with the high-end character of these service categories. The "surge_multiplier"-based splits also demonstrate how demand affects pricing tactics.

Although reasonable, the decision trees' accuracy—roughly 87.69% for Uber and 80.30% for Lyft—does indicate that there is potential for improvement, particularly when taking into account the heterogeneity of the MSE among nodes. Some nodes' more significant MSE indicates feature space regions where the model's predictions are less accurate.

The decision rules and thresholds that the models employ to generate predictions are visible upon visual inspection of the tree plots. For instance, distance is a recurrent split variable for both Uber and Lyft, highlighting its significance in price.

In conclusion, the decision trees offer an interpretable model that highlights the data-driven decision rules and the hierarchical significance of the variables. It might be beneficial to take into account ensemble techniques like Random Forests or boosting to increase prediction accuracy and model robustness, as well as perhaps incorporating cross-validation to fine-tune model complexity and avoid overfitting, given the complexity of the pricing structure, as indicated by the variety of splits.

6.3 Random Forest Model

The `randomForest` function was used to create random forests, an ensemble of decision trees, with a predetermined number of trees (`ntree = 100`) and parameters defined to measure feature importance.

For Uber:

The construction of the Uber random forest model (`uber_rmforest_model`) and its summary shed light on the performance of the ensemble. The same statistical analyses used for the earlier models were used to make predictions and assess their correctness.

For Lyft:

In a similar manner, the Lyft random forest model (`lyft_rmforest_model`) was trained and assessed. The goal of combining predictions from many trees was to increase the predictive performance's resilience and lower its variation.

```
[ ]: ### Random Forest

#Uber
#head(uberTraining)
uber_rmforest_model = randomForest(price ~., data = uberTraining, importance = TRUE, ntree = 100)
summary(uber_rmforest_model)

#prediction
```

```

uberPrediction_rmforest = predict(uber_rmforest_model, uberTesting[,-19])

#Correlation Matrix
actuals_predicts <- data.frame(cbind(actuals=uberTesting$price,
  ↳predicted=uberPrediction_rmforest))
correlation_accuracy <- cor(actuals_predicts)
correlation_accuracy

#Evaluation
mat_rf_uber<- regr.eval(uberTesting[,19], uberPrediction_rmforest)#, stats =
  ↳c('mape', 'rmse'))
print(mat_rf_uber)
errors = abs(uberPrediction_rmforest - uberTesting$price)
mape = 100 * (errors / uberTesting$price)
uber_rf_accuracy = 100 - mean(mape)
sprintf("The Accuracy of Random Forest for Uber :%f",uber_rf_accuracy)

#-----
#-----

lyft_rmforest_model = randomForest(price ~., data = lyftTraining, importance =
  ↳TRUE, ntree = 100)
summary(lyft_rmforest_model)

#prediction
lyft_pred_rmforest = predict(lyft_rmforest_model, lyftTesting[,-19])

#Correlation Matrix
actuals_predicts <- data.frame(cbind(actuals=lyftTesting$price,
  ↳predicted=lyft_pred_rmforest))

```



```

sprintf("The Accuracy of Random Forest for Uber :%f",uber_rf_accuracy)

#-----
#-----

lyft_rmforest_model = randomForest(price ~., data = lyftTraining, importance =
  TRUE, ntree = 100)
summary(lyft_rmforest_model)

#prediction
lyft_pred_rmforest = predict(lyft_rmforest_model, lyftTesting[,-19])

#Correlation Matrix
actuals_predicts <- data.frame(cbind(actuals=lyftTesting$price,
  predicted=lyft_pred_rmforest))
correlation_accuracy <- cor(actuals_predicts)
correlation_accuracy

#Evaluation
mat_rf_lyft<- regr.eval(lyftTesting[,19], lyft_pred_rmforest)#, stats =
  c('mape','rmse'))
print(mat_rf_lyft)

errors = abs(lyft_pred_rmforest - lyftTesting$price)n
mape = 100 * (errors / lyftTesting$price)
lyft_rf_accuracy = 100 - mean(mape)
sprintf("The Accuracy of Random Forest for Lyft :%f",lyft_rf_accuracy)

```

	Length	Class	Mode
call	5	-none-	call
type	1	-none-	character

```

predicted      297510 -none- numeric
mse            100 -none- numeric
rsq            100 -none- numeric
oob.times      297510 -none- numeric
importance     36 -none- numeric
importanceSD    18 -none- numeric
localImportance 0 -none- NULL
proximity      0 -none- NULL
ntree           1 -none- numeric
mtry           1 -none- numeric
forest         11 -none- list
coefs          0 -none- NULL
y              297510 -none- numeric
test           0 -none- NULL
inbag          0 -none- NULL
terms          3 terms  call

```

A matrix: 2 × 2 of type dbl		actuals	predicted
	actuals	1.0000000	0.9592433
	predicted	0.9592433	1.0000000

```

      mae      mse      rmse      mape
1.7313295 6.5205389 2.5535346 0.1290741

```

'The Accuracy of Random Forest for Uber :87.092593'

```

      Length Class  Mode
call           5 -none- call
type           1 -none- character
predicted      276667 -none- numeric
mse            100 -none- numeric
rsq            100 -none- numeric
oob.times      276667 -none- numeric
importance     36 -none- numeric
importanceSD    18 -none- numeric
localImportance 0 -none- NULL

```

```

mae      mse      rmse      mape
terms      3 terms  call

```

A matrix: 2 × 2 of type dbl		actuals	predicted
	actuals	1.0000000	0.9592433
	predicted	0.9592433	1.0000000

```

mae      mse      rmse      mape
1.7313295 6.5205389 2.5535346 0.1290741

```

'The Accuracy of Random Forest for Uber :87.092593'

```

Length Class Mode
call      5 -none- call
type      1 -none- character
predicted 276667 -none- numeric
mse       100 -none- numeric
rsq       100 -none- numeric
oob.times 276667 -none- numeric
importance 36 -none- numeric
importanceSD 18 -none- numeric
localImportance 0 -none- NULL
proximity 0 -none- NULL
ntree     1 -none- numeric
mtry      1 -none- numeric
forest    11 -none- list
coefs     0 -none- NULL
y         276667 -none- numeric
test      0 -none- NULL
inbag     0 -none- NULL
terms     3 terms  call

```

A matrix: 2 × 2 of type dbl		actuals	predicted
	actuals	1.0000000	0.9775778
	predicted	0.9775778	1.0000000

```

mae      mse      rmse      mape
1.666995 5.217596 2.284206 0.128877

```

'The Accuracy of Random Forest for Lyft :87.112304'

An ensemble of decision trees is used by the Random Forest model to forecast the price variable when it is applied to pricing data from Uber and Lyft. According to the result, 297,510 observations for Uber and 276,667 observations for Lyft were used to train the model.

Model Complexity and Performance Metrics:

1. Number of Trees: For both the Uber and Lyft models, the default value (ntree) is 100 trees, which is common and typically offers a reasonable compromise between computational efficiency and prediction accuracy.

2. Mean Absolute Error (MAE): This statistic calculates the average size of a set of forecasts' errors without taking into account their direction. More accurate forecasts are shown by lower MAE values. The MAE for Uber and Lyft, according to your results, is 1.7313 and 1.6669, respectively.

3. Mean Squared Error (MSE): The average squared difference between the estimated and actual values is shown by the Mean Squared Error (MSE). Lyft's MSE is 5.2176, while Uber's is 6.5205. In general, a model that fits the data better has a lower MSE.

4. Root Mean Squared Error (RMSE): The sample standard deviation of the variations between expected and actual values is represented by the square root of the mean squared errors. The RMSEs for Lyft and Uber are 2.2842 and 2.5535, respectively, indicating that Lyft's model is marginally more accurate.

5. Mean Absolute Percentage Error (MAPE): Accuracy is expressed as a percentage of error using the Mean Absolute Percentage Error (MAPE). The MAPE is 12.9074% for Uber and 12.8877% for Lyft. This shows that the model's forecasts typically match the actual prices by about 13%.

6. Accuracy: We get 87.0925% for Uber and 87.1123% for Lyft when we subtract the MAPE from 100. This implies that the Lyft model is somewhat more accurate than the other model, with both models performing at comparable levels.

Variable Importance:

We can determine which factors have the biggest effects on price prediction based on the importance measure. Although your letter unfortunately omits the precise variable importances, in general, this metric would enable us to determine which features have the greatest influence on the fare projection.

Out-Of-Bag (OOB) Error:- By assessing the model using training data that was not used (out-of-bag) during the model's training process, the OOBError approach quantifies the prediction error of random forests, decision trees, and other machine learning algorithms. Nevertheless, your output does not include this measure.

In conclusion, the Random Forest model offers a strong predictive instrument for pricing forecasts with a reasonable degree of accuracy. By averaging several deep decision trees that were trained on

various portions of the same training set, it can reduce overfitting and increase forecast accuracy. This is where its value lies. The MSE and MAE are comparatively low show that the model is dependable and consistently performs well in estimating the cost of rides for both Uber and Lyft, with Lyft exhibiting marginally superior outcomes according to the criteria supplied.

7 Model Validation

```
[ ]: ## Model Evaluation

# Uber
print("----- Uber Statitics -----")
tab <-
  matrix(c(mat_lr_uber["mae"],mat_dt_uber["mae"],mat_rf_uber["mae"],mat_lr_uber["mse"],mat_dt_
  uber_lr_accuracy,uber_dt_accuracy,uber_rf_accuracy), ncol=3, byrow=TRUE)
colnames(tab) <- c("Linear Regression",'Decision Tree','Random Forest')
rownames(tab) <- c('MAE','MSE','RMSE','MAPE','Accuracy')
uber_tab <- as.table(tab)
uber_tab

print("----- lyft Statitics -----")
# Lyft
tab <-
  matrix(c(mat_lr_lyft["mae"],mat_dt_lyft["mae"],mat_rf_lyft["mae"],mat_lr_lyft["mse"],mat_dt_
  lyft_lr_accuracy,lyft_dt_accuracy,lyft_rf_accuracy), ncol=3, byrow=TRUE)
colnames(tab) <- c("Linear Regression",'Decision Tree','Random Forest')
rownames(tab) <- c('MAE','MSE','RMSE','MAPE','Accuracy')
lyft_tab <- as.table(tab)
lyft_tab

[1] "----- Uber Statitics -----"

      Linear Regression Decision Tree Random Forest
MAE      1.6697108      1.7995829      1.7313295
MSE      5.8347082      6.8132767      6.5205389
RMSE      2.4155141      2.6102254      2.5535346
MAPE      0.1191017      0.1230596      0.1290741
Accuracy  88.0898340      87.6940363      87.0925928

[1] "----- lyft Statitics -----"
```

```

←uber_lr_accuracy,uber_dt_accuracy,uber_rf_accuracy), ncol=3, byrow=TRUE)
colnames(tab) <- c("Linear Regression", 'Decision Tree', 'Random Forest')
rownames(tab) <- c('MAE', 'MSE', 'RMSE', 'MAPE', 'Accuracy')
uber_tab <- as.table(tab)
uber_tab

print("----- lyft Statitics -----")
# Lyft
tab <-
  ←matrix(c(mat_lr_lyft["mae"],mat_dt_lyft["mae"],mat_rf_lyft["mae"],mat_lr_lyft["mse"],mat_dt
  ←lyft_lr_accuracy,lyft_dt_accuracy,lyft_rf_accuracy), ncol=3, byrow=TRUE)
colnames(tab) <- c("Linear Regression", 'Decision Tree', 'Random Forest')
rownames(tab) <- c('MAE', 'MSE', 'RMSE', 'MAPE', 'Accuracy')
lyft_tab <- as.table(tab)
lyft_tab

```

```
[1] "----- Uber Statitics -----"
```

	Linear Regression	Decision Tree	Random Forest
MAE	1.6697108	1.7995829	1.7313295
MSE	5.8347082	6.8132767	6.5205389
RMSE	2.4155141	2.6102254	2.5535346
MAPE	0.1191017	0.1230596	0.1290741
Accuracy	88.0898340	87.6940363	87.0925928

```
[1] "----- lyft Statitics -----"
```

	Linear Regression	Decision Tree	Random Forest
MAE	1.8068009	2.6463307	1.6669953
MSE	6.2327136	12.5978293	5.2175959
RMSE	2.4965403	3.5493421	2.2842057
MAPE	0.1493054	0.1969981	0.1288770
Accuracy	85.0694620	80.3001895	87.1123040

We will examine and contrast the performance of each model with an emphasis on its statistical significance, error metrics, and potential biases or risks, based on the thorough results provided for Uber and Lyft pricing predictions using Linear Regression, Decision Tree, and Random Forest models.

Linear Regression Model Analysis:

Uber:

- **MSE:** 5.8347, an estimator quality metric; a lower value indicates superior quality.
- **RMSE:** The residuals' standard deviation is moderate, according to the RMSE of 2.4155.
- **MAE:** The average absolute difference between observed and anticipated prices is shown by MAE:1.6697.
- **MAPE:** 11.9017%, which shows that forecasts are typically 11.9% off from the actual price.
- **Predictive accuracy** is strong, as evidenced by the accuracy of 88.08934%.

Lyft:

- **MAE:** 1.8068, which is somewhat higher than Uber's MAE and suggests less accuracy.
- The **MSE**, which is greater than Uber's at 6.2327, indicates that the residuals have more volatility.
- **RMSE:** 2.4965, which is larger than Uber's RMSE and suggests a wider residual dispersion.
- **MAPE:** 14.93054%, which is higher than Uber's, suggests that the model is less accurate.
- The **accuracy**, which is lower than Uber's at 85.0694620%, indicates a poorer fit to the data.

Observations: In every indicator, the Uber linear regression model performs better than the Lyft one, indicating that the model is more adept at capturing the connection between Uber's features and pricing. Lyft's higher MSE and RMSE suggest that the pricing structure is more variable than the linear model can adequately represent.

Decision Tree Model Analysis:

Uber:

- The Linear Regression model's **MAE** of 1.7995 indicates that the prediction error levels are comparable.
- A larger range of errors is shown by the **MSE** of 6.8132, which is higher than that of linear regression.
-
- **RMSE:** 2.6102, which is greater once more and indicates more substantial prediction mistakes.
- **MAPE:** 12.30596%, which is a little higher percentage error than Linear Regression.
- **Accuracy:** 87.6940363%, which is marginally less than that of linear regression and indicates a lower level of predictive accuracy

Lyft:

- A less accurate model is shown by the **MAE:** 2.6463, which is greater than Uber's.
- **MSE:** 12.5978, which is much higher than Uber's and indicates less precise forecasts.
- **RMSE:** 3.5493, which is higher once more and indicates more average prediction errors.
- **MAPE:** 19.69981%, indicating a higher average percentage error and significantly higher than Uber's.
- With an **accuracy** of 80.3001895%, which is far lower than Uber's, the model appears to be less dependable for the Lyft app.

Observations: With greater error metrics and poorer accuracy, the Decision Tree model has been demonstrated to be less successful than Linear Regression. Poor generalization on

unseen data may result from overfitting, in which the Decision Tree may be incorporating noise into the model.

Random Forest Model Analysis:

Uber:

- Good average precision is indicated by the **MAE** of 1.7313.
- **MSE**: 6.5205, which shows that the forecasts fit the data well.
- **RMSE**: 2.5535, indicating significantly less precision and a tiny increase above Linear Regression.
- **MAPE**: 12.90741%, which shows that forecasts and actual prices are reasonably close.
- **Accuracy**: 87.0925928%, which is reasonably high overall but marginally lower than linear regression.

Lyft:

- **MAE**: 1.6669, which is slightly better than Linear Regression.
- Better fit is indicated by the **MSE** of 5.2175, which is lower than the Decision Tree.
- Lyft's **RMSE** of 2.2842 is the lowest of all the models, indicating more accurate forecasts.
- **MAPE**: 12.8877%, indicating that the forecasts and actual values are reasonably near.
- **Accuracy**: 87.1123040%, which is marginally superior to Uber's Random Forest model and indicates good reliability.

Observations: With a generally high accuracy level and less overfitting than the Decision Tree model, the Random Forest model finds a compromise between bias and variance. Improved prediction accuracy and generalization on unknown data are the results of integrating the resilience of averaging numerous decision trees.

Summary:

According to the investigation, Linear Regression offers a reliable baseline for Uber pricing forecasts with low error metrics and good accuracy. The Random Forest model appears to be marginally more accurate and dependable for Lyft. Especially for Lyft, decision trees seem to overfit and perform worse than the other models in terms of generalization.

Biases/Risks:

- Predictions may be biased due to the assumption that characteristics and the target have a linear relationship, which is not necessarily true.
- Decision trees are sensitive because they can overfit, particularly when dealing with noisy data.

- Random Forest tackles certain overfitting problems, but it can still have trouble with extremely noisy data and can be computationally costly. Random Forest adapts to changes in the data.

8 Conclusion

With each model displaying distinct advantages and disadvantages, the predictive modeling for Uber and Lyft pricing utilizing Linear Regression, Decision Tree, and Random Forest models has produced informative results.

Positive results: Particularly when it came to predicting Uber prices, linear regression showed excellent accuracy with comparatively low error metrics. This implies that the price and the independent variables have a strong linear relationship.

- Lyft received the greatest results with **Random Forest**, which demonstrated strong predictive power with the lowest error metrics and highest accuracy.
- Although they don't perform any better than the other models, **decision trees** provide interpretability, which is useful for comprehending how decisions are made.

Negative Results: Decision trees had a propensity to overfit, especially when it came to Lyft, which resulted in lesser accuracy and greater error metrics than the other models.

- The greater error metrics compared to Uber suggest that, despite its strong performance, linear regression may oversimplify the relationship for Lyft.

Recommendations: Linear regression appears to be a dependable option for a fast and efficient model for Uber pricing predictions.

- Random Forest is suggested for Lyft because of its exceptional accuracy and capacity to manage intricate, non-linear interactions in the data.
- To further increase accuracy, a hybrid strategy or ensemble approaches may be taken into consideration, which would combine the advantages of several models.

Caveats/Cautions:- Because of its sensitivity to non-linear interactions, linear regression could miss some subtleties in the price structure. It is important to maintain routine checks for the linearity assumption.

- To prevent overfitting, decision trees need to be carefully adjusted. In order to preserve the generalizability of the model, pruning and maximum depth setting may be required.

- Random Forest models have the potential to get complicated and demand a lot of processing power. It's critical to strike a balance between the depth and number of trees, the computational capacity at hand, and the requirement for prompt forecasts.
- As pricing plans and market conditions change, predictive performance should be regularly observed. As new information becomes available, what is effective today could not be in the future.

In order to guarantee continuous performance throughout time, stakeholders should be informed of the models' limitations and the significance of routine model maintenance.

9 Data Sources

Specific data set: <https://www.kaggle.com/datasets/brllrb/uber-and-lyft-dataset-boston-ma>

10 Source Code

Github link:- https://github.com/asarath12/DPA_Project

11 Bibliography

- H., Shashank. "Data Analysis of Uber and Lyft Cab Services." International Journal of Interdisciplinary Innovative Research & Development (IJIIRD) ISSN: 2456-236X, 2020. <http://ijiird.com/wp-content/uploads/050144.pdf>.
- Sun, Jingyu. "Comprehensive Analysis of Ride-Hailing Evidence from Uber and Lyft." Proceedings of the 2022 2nd International Conference on Financial Management and Economic Transition (FMET 2022), December 14, 2022, 370–80. https://doi.org/10.2991/978-94-6463054-1_41.
- Jiang, Shan, Le Chen, Alan Mislove, and Christo Wilson. "On Ridesharing Competition and Accessibility." Proceedings of the 2018 World Wide Web Conference on World Wide Web- WWW '18, April 2018. <https://doi.org/10.1145/3178876.3186134>. Schaller, Bruce. "Can Sharing a Ride Make for Less Traffic? Evidence from Uber and Lyft and Implications for Cities." Transport Policy 102 (March 2021): 1–10. <https://doi.org/10.1016/j.tranpol.2020.12.015>.
- Brodeur, Abel, and Kerry Nield. "An Empirical Analysis of Taxi, Lyft and Uber Rides: Evidence from Weather Shocks in NYC." Journal of Economic Behavior & Organization 152 (August 2018): 1–16. <https://doi.org/10.1016/j.jebo.2018.06.004>.
- https://www.researchgate.net/publication/369691034_Comprehensive_Analysis_of_Ride-Hailing_Evidence_from_Uber_and_Lyft
- <https://dl.acm.org/doi/fullHtml/10.1145/3178876.3186134>
- https://itspubs.ucdavis.edu/download_pdf.php?id=2851